

DevBench: An Interaction-Grounded Benchmark for Code Completion Models

Anonymous Authors¹

Abstract

Code completion is the most frequent form of human–AI interaction in modern software development: a developer positions their cursor, and the model suggests what comes next. Yet existing benchmarks for this critical interaction are not grounded in how developers actually use completion tools. **DevBench** is an interaction-aware benchmark derived from over **one billion real developer code completion interactions**, capturing the scenarios, cursor positions, and difficulty patterns where current models succeed and fail in practice. It includes 1,800 evaluation instances across six programming languages and six empirically derived task categories, synthesized using multiple generator model families to ensure contamination resistance. The evaluation combines functional correctness (Pass@1), similarity-based metrics, and LLM-judge assessments of contextual relevance and helpfulness, mirroring the distinction developers themselves make between “correct” and “useful” completions. Evaluating 9 state-of-the-art models, the strongest achieves only 43.5% Pass@1, and models that score highest on perceived helpfulness often fail hidden functional assertions, revealing a tension directly relevant to developer trust. Our benchmark provides fine-grained, cross-language diagnostics to guide improvement of the completion models that power everyday developer–AI collaboration.

1. Introduction

Code completion has become the primary interface between developers and AI in modern software development. Tools like GitHub Copilot (GitHub, 2025) and Cursor (AnySphere, 2025) are used millions of times daily, turning each cur-

sor placement into a moment of human–AI collaboration: the developer provides context through their code, and the model must infer intent, respect constraints, and produce a completion the developer will accept. As this interaction becomes central to how software is written, realistic evaluation frameworks grounded in actual developer experience, not synthetic puzzles, are essential to understanding where these systems help and where they fall short.

Existing benchmarks evaluate different aspects of code generation: problem solving benchmarks for coding problems (Chen et al., 2021; Austin et al., 2021; Hendrycks et al., 2021; Iyer et al., 2018; Yin et al., 2018), repository-based benchmarks for challenges in large projects (Wu et al., 2024; Du et al., 2023; Ding et al., 2023; Yu et al., 2024; Zhuo et al., 2025; Jimenez et al., 2024; Deng et al., 2025), and evolving benchmarks addressing contamination (Li et al., 2024; Jain et al., 2024).

However, existing benchmarks rely on code samples scraped from open source repositories or coding challenge websites and generate target completions based on static rules for filling in line, function, or class implementations. This limits them in several ways: First, the target completions are not based on real world usage patterns for code completion tools, and therefore do not focus on common challenging completion scenarios that arise in real world usage. Second, the diagnostic value of these benchmarks is limited because they report aggregate metrics, but cannot attribute differences in performance to specific usage areas. Third, benchmarks collected from publicly available sources are prone to training data contamination, which has been observed in models overfitting to existing benchmarks (Jain et al., 2024). These limitations are especially consequential for code completion, an inherently interactive task where the developer immediately accepts, edits, or rejects the model’s output.

To address these limitations, we introduce **DevBench**, a realistic and scalable benchmark grounded in observed developer behavior. **DevBench** focuses on common yet challenging completion scenarios, identified from internal telemetry of over **one billion developer code completion interactions** and synthesized into 1,800 evaluation instances spanning six languages and six empirically distinct task categories (mean pairwise $\bar{\rho}$ =0.44), using generator models

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

from multiple provider families with validated cross-family fairness ($\rho=0.91$ ranking consistency). Each instance is reviewed for quality and realism, ensuring that tasks reflect how developers actually use code completion tools while remaining contamination-resistant.

As shown in Table 1, **DevBench** advances beyond existing benchmarks in both realism (Paul et al., 2024) and scope. It offers five key advantages: (1) **realism**, with tasks rooted in observed developer behavior; (2) **contamination resistance**, through synthetic but controlled instance generation; (3) **fine-grained evaluation**, assessing semantic alignment and developer utility; (4) **cross-language coverage**, spanning Python (Py), JavaScript (JS), TypeScript (TS), Java, C++, and C#; and (5) **interaction-grounded design**, with categories and difficulty derived from real developer completion interactions.

Together, these features provide ecological validity: **DevBench** reflects **authentic developer challenges** rather than hypothetical tasks, is validated through expert review, and captures diverse contexts across languages and developer skill levels. By enabling both overall rankings and scenario-specific diagnostics, **DevBench** supports informed model selection, targeted optimization, and deeper understanding of how completion models perform in the interaction scenarios developers actually encounter. We open-source the 1,800-instance benchmark and evaluation code.¹

2. Benchmark Design

We view code generation as a composite, puzzle-solving task in which models must combine distinct capabilities, such as API usage, intent understanding, and syntax control. To evaluate these skills, we define benchmark categories that isolate each capability while ensuring every instance is solvable from the provided prefix/suffix, making evaluation both realistic and fair. Although individual instances are synthesized, **DevBench** is telemetry-driven: its categories, task types, and scenarios are derived from analysis of over one billion real developer interactions, with synthesis used only to instantiate these empirically derived patterns in a privacy-preserving, contamination-resistant manner.

2.1. From User Telemetry to Categories

DevBench is derived from an internal telemetry corpus of over one billion anonymized code-completion interactions, including prefix/suffix context, generated completions and final accepted completions, and user actions (accept, reject, edit), the direct outcome of each developer-model interaction. This dataset spans diverse contexts over IDEs, geographical locations, language distribution, and develop-

ers ranging from students to senior engineers. To satisfy privacy and compliance requirements, we do not release or reuse raw user code; all benchmark instances are synthetic.

Category derivation used a three-stage telemetry analysis (Appendix A.1). We first drew a stratified, difficulty-enriched sample of 50,000 completions, balanced across the six target languages and enriched for low-acceptance or high-edit-distance cases, i.e., the interactions where current completion models most frequently fail to meet developer expectations. Three researchers then annotated 3,000 completions (500 per language) with scenario and failure-mode labels, achieving substantial agreement on the primary label (Fleiss’ $\kappa = 0.74$ before adjudication). The resulting clusters were consolidated into six task categories retained only if they satisfied *prevalence* (appearing in $\geq 5\%$ of low-acceptance completions across at least four languages) and *actionability* (isolable into a self-contained prefix/suffix completion task). Because the telemetry distribution is not uniform across languages, the released benchmark is balanced rather than proportional: 300 instances per language and 50 per language-category cell, prioritizing statistical power for cross-language diagnostics. **DevBench** is therefore a telemetry-grounded, balanced diagnostic benchmark rather than a proportional sample of all completion traffic.

2.2. Benchmark categories

We define six benchmark categories based on our analysis of user telemetry. Each category targets a distinct type of developer intent (a different reason a developer places their cursor and requests a completion) and is consistently evaluated across languages, with adaptations to reflect the idioms and ecosystems of each target language (see Table 2). The categories are described in detail below (examples in Appendix D.1).

1. API Usage: This category tests a model’s ability to correctly apply specialized library functions. Each evaluation instance consists of a prefix that sets the context, a golden completion illustrating proper API usage, and a suffix for continuation.

2. Code Purpose Understanding: This category evaluates whether a model can infer business logic, domain conventions, and state constraints from surrounding code, rather than merely producing syntactically valid completions. For instance, given a `Subscription` class with state transitions, an audit log, and a configurable cooldown period, the model must implement a `resume` method that checks cooldown elapsed time, verifies the current state is `paused`, and reuses the existing `_transition` helper for audit-log consistency.

3. Code2NL/NL2Code: This category evaluates a model’s ability to translate between code and natural language (NL)

¹Code and benchmark: https://anonymous.4open.science/r/devbench_dl4c_submission-E465

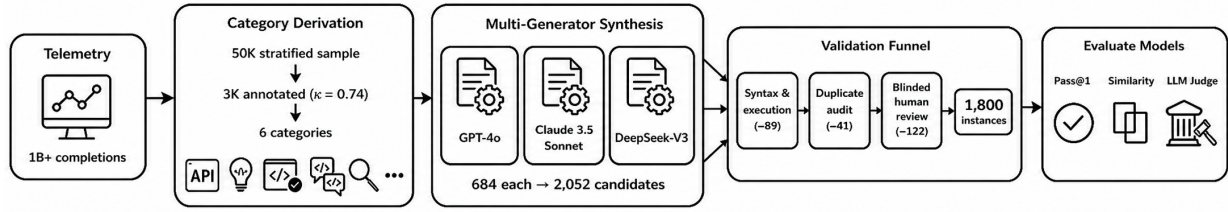


Figure 1. End-to-end **DevBench** pipeline. Telemetry analysis of 1B+ completions produces six task categories via stratified sampling, annotation ($\kappa = 0.74$), and clustering. Three generator models from different provider families (GPT-4o, Claude 3.5 Sonnet, DeepSeek-V3) each produce 684 candidate instances. A three-stage validation funnel (automatic syntax/execution checks, near-duplicate filtering against public benchmarks, and blinded human review) reduces 2,052 candidates to the final 1,800 instances. Models are evaluated on functional correctness (Pass@1), similarity metrics, and LLM-judge assessments.

Table 1. Comparison of recent code generation benchmarks across size, language coverage, focus, source, and unique features.

Benchmark	# Tasks	Languages	Focus	Source	Unique Feature
RepoMasterEval	288	Py, TS	Real-world repo completion	GitHub repos (>100 stars)	Mutation testing for test robustness
CrossCodeEval	~10k	Py, Java, TS, C#	Cross-file dependencies	GitHub repos (>3 stars)	Static analysis for dependencies
CoderEval	460	Py, Java	Cross-file pragmatic generation	GitHub repos (popular tags)	Human-labeled docstrings
ClassEval	100	Py	Class-level generation	Manually crafted	Multiple interdependent methods
HumanEval	164	Py	Basic programming tasks	Manually crafted	Simple interview-style problems
HumanEval+	164	Py	Enhanced testing rigor	Manually crafted	80× more evaluation instances
LiveCodeBench	511	Py	Contamination-free evaluation	Competition platforms	Time-based contamination tracking
SWE-bench	2,294	Py	Repo-level bug fixing	GitHub issues and PRs	Real-world issues from 12 popular repos
BigCodeBench	1,140	Py	Diverse function calls as tools	Human-LLM collab. generation	723 function calls from 139 libs
DevBench (ours)	1,800	Py, JS, TS, Java, C++, C#	Realistic developer-informed scenarios	Synth. generated, manually reviewed	Telemetry-guided, human-validated

in both directions. This reflects real-world developer workflows, where boundaries between code and language are increasingly blurred. To align with practical use cases, our benchmark covers a wide spectrum of scenarios including: (1) NL only in the prefix, (2) code only in the prefix, (3) mixed NL and code in the prefix, (4) various NL forms including docstrings, inline comments, block comments, and user-facing documentation, and (5) different documentation styles across programming languages (e.g., Python docstrings, JavaDoc, JSDoc, XML docs, and Doxygen).

4. Low Context: This category evaluates a model’s ability to complete code using minimal context (10–20 lines total), requiring it to recognize language-specific patterns and idioms. These tasks are carefully designed to be solvable despite limited information, testing the model’s deep understanding of programming conventions without relying on broader context.

5. Pattern Matching: This category tests a model’s ability to recognize and extend established code patterns within

realistic contexts. The prefix establishes a pattern through examples or specifications, and the model must implement a structurally similar but semantically distinct continuation, following the intended structure rather than generating arbitrary code.

6. Syntax Completion: This category tests a model’s ability to generate complex, nested structures while adhering to language-specific syntax rules. Evaluation instances span four categories: nested control structures, complex features, multi-line patterns, and error handling. The model must correctly manage indentation, close code blocks, and match braces or parentheses, demonstrating mastery of each language’s unique syntactic constructs.

Category relationships. Because real developer tasks blend capabilities, we do not expect perfect category orthogonality. Empirically, pairwise Spearman correlations of Pass@1 across 54 model–language combinations have mean $\bar{\rho} = 0.44$, with low-correlation pairs such as API Usage vs. Code Purpose Understanding ($\rho = 0.16$) and API

Table 2. Language-specific adaptations. NL = Natural Language, HOFs = Higher-Order Functions, RAI = Resource Acquisition Is Initialization.

Category	Python	C#	C++	Java	JavaScript	TypeScript
API Usage	inspect, weakref, csv dialects	LINQ, Span, Reflection	Smart ptrs, concurrency, charconv	DateTimeFormatter, ConcurrentHashMap	Node.js crypto, Buffer, Atomics	Same as JS w/ types
Code Purpose	Iterators/ generators, context mgrs	LINQ/ collections, decimal arith.	Containers, struct workflows	Streams/ collections, BigDecimal	Closures, class state machines	Same as JS w/ type systems
Code2NL	Docstrings	XML docs	Doxygen comments	Javadoc	JSDoc	TSDoc w/ type annot.
Low Context	Decorators, context mgrs	Span, LINQ, yield	RAII, move semantics	Collections views, atomics	Proxy, WeakMap, HOFs	Type guards, branded types
Pattern Match.	Config parsers, template dispatch	Config parsers, edit distance	Line-continuation, bracket matching	Line-continuation, check digits	Template engines, deep equality	Escape-aware parsers, tokenizers
Syntax Compl.	Match/case, metaclasses, async	Switch exprs, records, LINQ	Fold exprs, constexpr if, RAI	Sealed classes, records, switch patterns	Async generators, Proxy, Symbols	Conditional types, overloads

Usage vs. Pattern Matching ($\rho = 0.17$). Conversely, Pattern Matching and Code Purpose Understanding correlate more highly ($\rho = 0.74$), yet still diverge diagnostically for individual models (e.g., DeepSeek V4 Pro excels at Pattern Matching but underperforms GPT-5.5 on Syntax Completion, 32.4% vs. 51.4%; Table 5). PCA shows the first two components explain $\approx 77\%$ of variance and three explain $\approx 87\%$, indicating two to three capability dimensions beyond a single code-generation factor. Full correlations are in Appendix C.3.

2.3. Benchmark construction

Figure 1 overviews the construction pipeline: starting from telemetry-derived categories, we generate the benchmark.

Evaluation Instance Structure: Each instance contains a prefix, a golden completion, an optional suffix for fill-in-the-middle (FIM) settings, and hidden assertions for functional validation. Assertions are never visible to the evaluated model, and cursor positions are chosen at natural code boundaries observed in telemetry, covering both prefix-only and FIM-style completions.

Generation and Validation: To mitigate single-generator stylistic bias, instances were synthesized using three generator models from different provider families: GPT-4o (OpenAI et al., 2024), Claude 3.5 Sonnet (Anthropic, 2024), and DeepSeek-V3 (DeepSeek-AI et al., 2025). These generators are earlier than the corresponding evaluated frontier models, reducing circularity between synthesis and evaluation. Each generator produced 684 candidates using the same structured prompts (Appendix A.5), yielding 2,052 candidates. After automatic syntax/execution validation, near-duplicate filtering against public benchmarks, and blinded human review, the final benchmark contains 1,800 instances with near-balanced generator provenance (GPT-4o: 597, Claude 3.5 Sonnet: 601, DeepSeek-V3: 602). Each instance was independently reviewed by two annotators from a team of three senior researchers and engineers

with expertise across all six target languages, evaluating four dimensions: (1) *usefulness* (plausible developer need), (2) *realism* (authentic coding patterns, including common inconsistencies), (3) *category alignment* (consistent with intended task type), and (4) *complexity authenticity* (genuine difficulty observed in telemetry). Inter-annotator agreement was substantial (Cohen’s $\kappa = 0.82$, 91% raw agreement); the most common rejection reasons were overly simplified or “textbook-perfect” implementations (32%), insufficient complexity (28%), unrealistic edge-case handling (23%), and category misalignment (17%). Full validation details are in Appendix A.2.

Complexity and Diversity: DevBench instances average 57.6 LOC with a balanced prefix/completion structure (117.9 prefix tokens, 108.2 completion tokens), as shown in Tables 3 and 4. In contrast, CrossCodeEval features long prompts (71–116 LOC) but extremely short completions (1–2 LOC); DevBench’s balance is more reflective of practical code-completion workflows. To verify that repeated use of the same language–category prompt did not produce homogeneous instances, we measured within-prompt Term Frequency–Inverse Document Frequency (TF-IDF) cosine similarity; the mean pairwise similarity is 0.094 (cosine distance 0.906), indicating high within-prompt diversity (Appendix A.4).

Generator Bias Analysis. We evaluated generator-family favoritism by fitting a logistic regression over model–instance pass/fail outcomes with fixed effects for evaluated model, generator family, language, category, and completion-length bin. The same-family coefficient was small and not statistically significant, and model rankings on each generator’s subset were highly correlated (minimum pairwise Spearman $\rho = 0.91$). We find no evidence of systematic generator-family favoritism. To reduce instance-level contamination risk, we also filtered candidates against the established benchmarks in Table 3 using token 4-gram Jaccard and AST structural similarity, removing 41 of 2,052 candidates (2.0%). DevBench is thus de-

Table 3. Avg. LOC of code generation benchmarks.

Benchmark	Avg. LOC
DevBench (ours)	57.6
CrossCodeEval	71–117
CoderEval-Py	32.0
APPS	21.4
HumanEval	11.5
CoderEval-Java	10.2
MBPP	6.8
Concode	4.8
DS-1000	3.8
CoNaLA	1.0

Table 4. DevBench language-specific statistics.

Language	Prefix LOC	Compl. LOC	Total LOC	Prefix Tok.	Compl. Tok.
Python	14.7	11.3	48.8	71.6	81.3
C#	20.2	12.4	58.5	121.3	106.2
C++	24.1	10.5	63.0	149.3	101.5
Java	16.2	10.2	51.0	107.3	91.0
JavaScript	15.6	12.0	62.3	112.2	119.4
TypeScript	19.8	13.7	62.1	145.4	149.7
Average	18.4	11.7	57.6	117.9	108.2

signed to be contamination-resistant at the instance level while reflecting general programming patterns that all code models are expected to learn; full methodology is in Appendix A.3.

3. Evaluation methods

Given the challenges in evaluating LLMs, we employ a combination of methods: functional correctness; similarity-based metrics, which offer fast, scalable evaluation across languages; and LLM-judge evaluations to assess output quality from a human-aligned perspective.

3.1. Functional correctness

For functional correctness, we report Pass@1 with $n = 5$ samples (Chen et al., 2021), measuring the probability that at least one generated sample passes all test cases: $\text{pass}@k := \mathbb{E}_{\text{Problems}}[1 - \binom{n-c}{k} / \binom{n}{k}]$ where c is the number of correct samples and $k = 1$. Execution details are in Appendix B.2.

3.2. Similarity-Based evaluation

We use two widely adopted similarity metrics: Average Cosine Similarity and Line 0 Exact Match Rate. Average Cosine Similarity assesses semantic equivalence across the full completion, even when syntax differs, while Line 0 Exact Match focuses on strict precision at the start of the completion. Each metric is averaged over the $n = 5$ generated completions per test case.

Average Cosine Similarity: We use token-based cosine similarity (Zhou et al., 2023) to measure semantic overlap between model-generated and golden completions. When tokenization fails due to unusual code constructs, we fall back to character n-grams (1-3) to ensure robust comparison.

Line 0 Exact Match Rate: We calculate the percentage of cases where the first line of the model-generated completion exactly matches the first line of the golden completion (Ding et al., 2023).

3.3. LLM-judge evaluation

We use an LLM judge to approximate the developer-facing perspective, i.e., whether a completion *looks right* in context (the perception driving accept/edit/reject decisions), scoring each completion on two 0–5 dimensions: **relevance** to the surrounding context and **helpfulness** in advancing the task, yielding a 0–10 combined score averaged over $n = 5$ completions per task. We use Gemini 2.5 Flash (Comanici et al., 2025) as judge because it belongs to a different provider family than all evaluated models and achieves strong code-judge accuracy on CodeJudgeBench (74.66) (Jiang et al., 2025); independent shortcut-bias audits report that Gemini 2.5 Flash exhibits smaller verdict shift rates under provenance and recency cue perturbations than GPT-4o (Mari Oriyad et al., 2025). We blind the judge to model identity and use point-wise scoring rather than pairwise comparison to avoid pairwise response-ordering effects.

We validate the judge against 450 human-scored completions stratified by language, model, category, and score range. Three annotators independently scored each completion, with acceptable reliability (Intraclass Correlation Coefficient, ICC(3,1) = 0.76); Gemini judgments correlate with averaged human scores at Spearman $\rho = 0.81$ ($p < 0.001$), with per-language correlations ranging from $\rho = 0.74$ (C#) to $\rho = 0.87$ (Python) and no language-level correlation falling below 0.70. Large judge–human disagreements occurred in 50 of 450 cases (11%); full calibration details and failure analysis are in Appendix B.4. Confidence intervals are estimated via 10,000 bootstrap resamples.

4. Experiments

4.1. Experimental setup

Models: We evaluated 9 state-of-the-art LLMs spanning six providers, three capability tiers, and both closed and open weights. Our selection includes three frontier models spanning closed and open availability (GPT-5.5 (OpenAI, 2026b), Claude Opus 4.7 (Anthropic, 2026a), and DeepSeek V4 Pro (DeepSeek-AI, 2026)); large open-weight models from Meta and Mistral (Llama 4 Maverick (Meta AI, 2025) and Mistral Medium 3.5 (Mistral AI, 2026)); and mid-tier or compact models: Claude Sonnet 4.6 (Anthropic, 2026b), GPT-5.4 Mini and GPT-5.4 Nano (OpenAI, 2026a), and Qwen3.6-27B (Qwen Team, 2026). This suite covers closed frontier systems, large open-weight models, and compact/open models across six providers, including four open-weight families: DeepSeek, Meta, Mistral, and

Alibaba.

Evaluation Setup: Following prior work (Jain et al., 2024), we set a maximum output length of 800 tokens to accommodate complex completions. Models that support temperature with reasoning enabled used a temperature of 0.2 for more deterministic completions; remaining models used their default reasoning settings. All models used nucleus sampling with top-p=1.0 to preserve the full token distribution while modulating randomness via temperature. For LLM-judge evaluation, we used Gemini 2.5 Flash (Comanici et al., 2025) via Vertex AI with default settings (temperature=1.0 and top-p=1.0). Models were evaluated in a zero-shot setting, each prompted using a consistent, code-only template, excluding explanations or comments. See Appendix B.3 for prompt details, Appendix B.1 for infrastructure, and Appendix E.2 for key assumptions.

4.2. Results and insights

4.2.1. FUNCTIONAL CORRECTNESS (PASS@1)

Table 5 shows Pass@1 results with $n = 5$ samples, averaged across the six programming languages. Language-specific Pass@1 breakdowns are provided in Table 8.

Top Performers: GPT-5.5 leads with 43.5%, followed closely by DeepSeek V4 Pro (43.3%) and Claude Opus 4.7 (40.5%). No model exceeds 44% Pass@1, confirming that **DevBench** remains challenging even for frontier reasoning models.

Open Models: Llama 4 Maverick (35.8%) and Mistral Medium 3.5 (32.1%) slot between the frontier and compact tiers. Llama outperforms Claude Sonnet 4.6 overall, while Mistral is within 0.5 percentage points of Sonnet and above the GPT-5.4 compact models, demonstrating that large open-weight models are competitive with closed mid-tier systems on hardened code completion tasks.

Small-Size Models: Qwen3.6-27B achieves a modest overall Pass@1 of 18.0%, while GPT-5.4 Nano reaches 27.1%. The roughly 25-point gap between the strongest and weakest model demonstrates clear discrimination across the capability spectrum, with a smooth gradient from frontier (40–44%) through mid-tier open and closed models (32–36%) to compact models (27–28%) and the smallest open model (18%).

Category Patterns: Low Context is the strongest category across models (top performers: 46–54%), while Code2NL/NL2Code is the most challenging (leading models: 29–32%, most below 26%). Pattern Matching shows the largest model differentiation, with DeepSeek V4 Pro at 60.4% and Qwen3.6-27B at 22.7%. API Usage is relatively uniform among the frontier cluster (33–38%) but drops sharply for smaller models (Qwen3.6-27B: 18.1%). These category-level patterns correspond to distinct interac-

tion failure modes that developers experience differently in practice.

4.2.2. SIMILARITY-BASED EVALUATION

Table 6 reports similarity metrics across languages, averaged over categories. Additional similarity results are in Appendix C.1; qualitative examples of the behaviors described below appear in Appendices D.2 and D.3.

Top Performers: DeepSeek V4 Pro, Llama 4 Maverick, and Mistral Medium 3.5 achieve the strongest average cosine similarity scores across most languages, indicating that their completions often remain close to the golden completions even when they fail hidden assertions. DeepSeek V4 Pro retains the highest Line 0 Exact Match Rates in Python and JavaScript (60.67% and 62.67%), while Mistral Medium 3.5 achieves the highest cosine similarity in Python and Java.

Metric Discrepancies: DeepSeek V4 Pro achieves the highest similarity scores overall despite being tied with GPT-5.5 on Pass@1 (43.3% vs 43.5%), suggesting its completions are syntactically closer to golden completions even when they may fail hidden assertions. Conversely, Claude Opus 4.7’s C# similarity collapse (0.12 cosine) aligns with its 9.9% Pass@1 in that language. This failure confirms a systematic language-specific gap rather than a metric artifact. This demonstrates how combining functional correctness with similarity metrics provides richer diagnostic insight than either metric alone.

Language-Specific Challenges: C# shows the lowest cosine similarity for several models, while TypeScript is lowest for others. C# is particularly challenging for Anthropic models (Opus 4.7: 0.12, Sonnet 4.6: 0.21). This language-specific variation highlights the importance of cross-language evaluation for identifying targeted improvement opportunities.

4.2.3. LLM-JUDGE EVALUATION

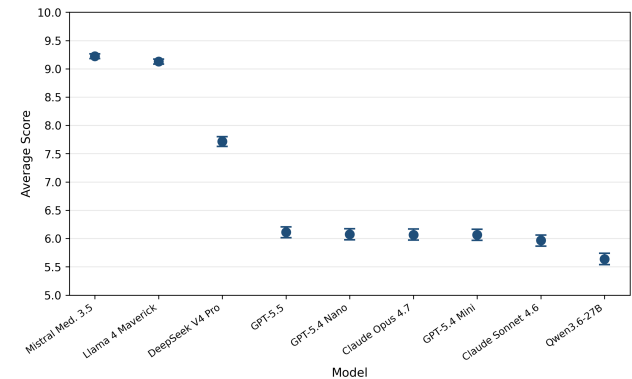


Figure 2. Overall LLM-judge evaluation scores with 95% confidence intervals.

Table 5. Pass@1 with $n = 5$ across code completion categories.

Model	Overall ↓	API Usage	Code Purpose	Code2NL/NL2Code	Low Context	Pattern Matching	Syntax
GPT-5.5	43.5%	37.5%	37.6%	31.5%	54.2%	48.8%	51.4%
DeepSeek V4 Pro	43.3%	34.5%	55.5%	29.3%	48.0%	60.4%	32.4%
Claude Opus 4.7	40.5%	33.8%	40.1%	31.5%	46.7%	45.4%	45.7%
Llama 4 Maverick	35.8%	30.6%	49.9%	22.5%	41.7%	40.6%	29.7%
Claude Sonnet 4.6	32.6%	31.9%	27.3%	27.1%	45.6%	27.5%	36.3%
Mistral Medium 3.5	32.1%	26.6%	45.5%	21.3%	34.8%	38.9%	25.7%
GPT-5.4 Mini	28.2%	32.7%	16.3%	25.5%	40.3%	25.3%	29.1%
GPT-5.4 Nano	27.1%	29.8%	15.4%	25.3%	41.1%	23.4%	27.7%
Qwen3.6-27B	18.0%	18.1%	16.1%	13.8%	26.3%	22.7%	11.3%

Table 6. Similarity metrics across programming languages.

Model	Average Cosine Similarity							Line 0 Exact Match Rate (%)						
	Py	JS	TS	Java	C++	C#		Py	JS	TS	Java	C++	C#	
GPT-5.5	0.64	0.62	0.53	0.67	0.58	0.44		54.00	53.67	32.33	51.00	39.00	43.33	
DeepSeek V4 Pro	0.72	0.72	0.69	0.73	0.69	0.61		60.67	62.67	39.00	56.67	45.00	56.00	
Claude Opus 4.7	0.63	0.58	0.56	0.70	0.61	0.12		53.67	49.33	35.33	54.67	39.00	14.33	
Llama 4 Maverick	0.71	0.70	0.66	0.70	0.67	0.67		53.67	47.67	26.67	47.33	37.67	42.33	
Claude Sonnet 4.6	0.52	0.44	0.42	0.58	0.49	0.21		43.00	37.33	26.67	47.00	31.67	22.67	
Mistral Med. 3.5	0.73	0.70	0.67	0.74	0.68	0.00		54.67	48.00	29.67	47.33	38.00	0.00	
GPT-5.4 Mini	0.47	0.38	0.30	0.49	0.46	0.48		45.33	38.00	20.67	43.67	34.00	41.67	
GPT-5.4 Nano	0.34	0.30	0.29	0.47	0.41	0.40		31.33	31.33	25.67	43.67	32.00	35.33	
Qwen3.6-27B	0.35	0.38	0.33	0.40	0.38	0.38		44.33	45.67	22.33	40.00	33.67	34.67	

Figure 2 presents the final LLM-judge scores with 95% confidence intervals and Appendix C.4 provides a detailed breakdown by category and language.

LLM-judge score patterns: The LLM-judge ranking differs substantially from Pass@1, reflecting that the judge scores relevance and helpfulness rather than executable correctness. Llama 4 Maverick and Mistral Medium 3.5 receive the highest judge scores (9.13 and 9.22 respectively) despite lower Pass@1, indicating that their completions are often contextually plausible under the relevance/helpfulness rubric, even when they fail hidden functional assertions. Among the frontier models, DeepSeek V4 Pro leads the judge at 7.71, ahead of GPT-5.5 (6.11) and Claude Opus 4.7 (6.07).

Pass@1 vs. Judge Divergence: GPT-5.5, which leads on Pass@1 (43.5%), scores below Llama 4 Maverick and Mistral Medium 3.5 on the judge, despite those models ranking lower on Pass@1. This divergence underscores that plausible-looking completions do not necessarily satisfy hidden assertions, reinforcing why **DevBench** reports functional correctness alongside judge assessments. Because the judge scores relevance and helpfulness rather than executable correctness, divergence from Pass@1 is expected: a completion may be contextually useful but fail a hidden assertion, or it may be executable but less helpful to a developer. This gap measures the difference between “code that works” and “code developers want,” a distinction central to the quality of the completion interaction.

Confidence Intervals: Most models display relatively narrow confidence intervals, indicating consistent performance across evaluation instances. The narrow intervals indicate that the judge-score differences are not driven by a small number of outlier completions.

4.3. Diagnostic Case Study: DeepSeek V4 Pro

DevBench’s multi-metric framework enables fine-grained diagnosis beyond aggregate rankings. To demonstrate its practical utility, we present a case study on DeepSeek V4 Pro, a useful diagnostic case because it is near the top on Pass@1 while exhibiting sharp category-specific divergences between similarity and functional correctness.

Syntax vs. Semantics: DeepSeek V4 Pro excels in Syntax Completion similarity in Table 7 (Average Cosine Similarity 0.71 vs. GPT-5.5’s 0.67 and Line 0 Exact Match Rate 54.33% vs. 53.33%) but underperforms in functional correctness of the same category in Table 5 (32.4% vs. 51.4% Pass@1). This pattern indicates heavier reliance on pattern memorization than true semantic understanding. Manual review of failure cases confirms that DeepSeek V4 Pro often produces code syntactically close to the golden solution but functionally incorrect.

Category-Level Patterns: Based on Table 7, the model demonstrates strong performance in Pattern Matching (0.78 vs. GPT-5.5’s 0.56) and Code Purpose (0.76 vs. 0.42), where both similarity and Pass@1 confirm genuine strength. However, Code2NL/NL2Code is the model’s weakest category

(29.3% Pass@1, cosine 0.56). This disparity reveals the model’s tendency to excel at extending established code patterns rather than deeply understand and generate code in semantically rich tasks requiring bidirectional translation between natural language and code.

Language-Specific Gaps: While DeepSeek V4 Pro is the most balanced model across languages (9.4-percentage-point spread), it shows notable underperformance in TypeScript (36.1%) relative to its strongest languages (Table 8). The LLM-judge scores reveal a sharper gap: 9.70 in C++ but only 5.42 in JavaScript, suggesting the model’s completions vary significantly in developer-perceived quality across language ecosystems.

Preserving Strengths: DeepSeek V4 Pro already excels in Pattern Matching and Code Purpose Understanding, areas that should be maintained during future fine-tuning to avoid catastrophic forgetting.

These insights translate to actionable training priorities: (1) emphasize syntactic precision during fine-tuning to close the gap between similarity and functional correctness, (2) increase Code2NL/NL2Code training examples to improve semantic understanding, (3) target TypeScript and JavaScript samples in the training mix to close language-specific gaps, and (4) maintain current strengths in Pattern Matching and Code Purpose.

4.4. Implications for Developer–AI Completion Interaction

We evaluate the atomic unit of developer–AI collaboration: the single-turn code completion. Because **DevBench** is derived from over one billion such interactions, our findings speak directly to the quality of this experience. Three patterns emerge with direct implications for how developers interact with completion tools.

First, the helpfulness–correctness divergence documented in Section 4.2.1 means developers face a concrete tradeoff: models that produce the most contextually plausible completions (Llama 4 Maverick: judge score 9.13) are not the most functionally correct (Pass@1: 35.8%), and vice versa (GPT-5.5: 43.5% Pass@1, judge 6.11). A developer using the first model will accept completions that *look right* but introduce subtle bugs; with the second, they receive correct code that may not match their intent. Second, cross-language gaps (e.g., Claude Opus 4.7: 54.1% in Python vs. 9.9% in C#) mean completion quality varies unpredictably across files in a single project, precisely the inconsistency that erodes developer trust. Third, each category captures a distinct interaction failure mode, from incorrect API calls developers accept on appearance to breakdowns in natural-language–code translation where assistance is most needed.

5. Related Work

Existing LLM coding evaluation spans three main areas. *Problem solving benchmarks* like HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), and APPS (Hendrycks et al., 2021) evaluate coding problems of varying difficulty, while Concode and CoNaLa focus on natural language to code translation (Iyer et al., 2018; Yin et al., 2018). *Repository-based benchmarks* evaluate code generation within existing codebases, from simple masking tasks (RepoMasterEval, ClassEval (Wu et al., 2024; Du et al., 2023)) to inter-file reasoning (CrossCodeEval, CoderEval (Ding et al., 2023; Yu et al., 2024)), API usage (BigCodeBench (Zhuo et al., 2025)), and agentic problem solving (SWE-Bench (Jimenez et al., 2024), SWE-Bench Pro (Deng et al., 2025)). *Evolving benchmarks* like LiveCodeBench and EvoCodeBench address data contamination using recent code (Jain et al., 2024; Li et al., 2024). Recent work has also scaled multi-language evaluation: M2RC-Eval (Yin et al., 2025) provides repository-level code completion across 18 languages, and AutoCodeBench (Li et al., 2025) generates 3,920 contest-style problems across 20 languages. Both focus on standalone code generation rather than the FIM completion scenarios that arise in real developer workflows. In contrast, **DevBench** evaluates scenarios arising during live development, with task categories derived from telemetry analysis of real developer interactions. While agentic benchmarks like SWE-Bench evaluate autonomous, multi-step task completion, code completion remains the most frequent form of developer–AI interaction in practice, and **DevBench** complements agentic evaluation by providing fine-grained diagnostics grounded in real completion interactions.

6. Conclusion

We introduced **DevBench**, a benchmark grounded in over one billion real developer code completion interactions, enabling fine-grained, realistic evaluation across six languages and six task categories. Our results reveal a consistent tension between functional correctness and developer-perceived helpfulness, the same tension developers navigate each time they accept or reject a completion. Cross-language and cross-category diagnostics expose sharp capability gaps even in frontier models, gaps that directly affect the quality of the developer’s interaction experience. These findings underscore the need for interaction-grounded benchmarks that diagnose specific failure modes in realistic completion scenarios rather than reporting aggregate pass rates on synthetic tasks. By releasing the benchmark, we aim to support the development of completion models that are reliably helpful across the diverse scenarios arising in everyday coding. Future work could extend **DevBench** to multi-turn interactions and developer feedback.

References

- Anthropic. Introducing Claude 3.5 Sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>, 2024.
- Anthropic. Introducing Claude Opus 4.7. <https://www.anthropic.com/news/claude-opus-4-7>, April 2026a.
- Anthropic. Introducing Claude Sonnet 4.6. <https://www.anthropic.com/news/claude-sonnet-4-6>, February 2026b.
- AnySphere. Cursor - The AI Code Editor, 2025. URL <https://www.cursor.com/>.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., et al. Program Synthesis with Large Language Models, August 2021.
- Chen, H., Waheed, A., Li, X., Wang, Y., Wang, J., Raj, B., and Abdin, M. I. On the Diversity of Synthetic Data and its Impact on Training Large Language Models, October 2024.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., et al. Evaluating Large Language Models Trained on Code, July 2021.
- Comanici, G. et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, July 2025.
- DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf, April 2026.
- DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., et al. DeepSeek-V3 Technical Report, February 2025.
- Deng, X., Da, J., Pan, E., He, Y. Y., Ide, C., Garg, K., Lauffer, N., Park, A., Pasari, N., Rane, C., Sampath, K., Krishnan, M., Kundurthy, S., Hendryx, S., Wang, Z., Zhang, C. B. C., Jacobson, N., Liu, B., and Kenstler, B. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?, September 2025.
- Ding, Y., Wang, Z., Ahmad, W. U., Ding, H., Tan, M., Jain, N., et al. CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion, November 2023.
- Du, X., Liu, M., Wang, K., Wang, H., Liu, J., Chen, Y., et al. ClassEval: A Manually-Crafted Benchmark for Evaluating LLMs on Class-level Code Generation, August 2023.
- GitHub. GitHub Copilot: Your AI pair programmer, 2025. URL <https://github.com/features/copilot>.
- Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., et al. Measuring Coding Challenge Competence With APPS, November 2021.
- Iyer, S., Konstas, I., Cheung, A., and Zettlemoyer, L. Mapping Language to Code in Programmatic Context, August 2018.
- Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code, June 2024.
- Jiang, H., Chen, Y., Cao, Y., Lee, H.-y., and Tan, R. T. CodeJudgeBench: Benchmarking LLM-as-a-judge for coding tasks. *arXiv preprint arXiv:2507.10535*, 2025.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, November 2024.
- Li, J., Li, G., Zhang, X., Dong, Y., and Jin, Z. EvoCodeBench: An Evolving Code Generation Benchmark Aligned with Real-World Code Repositories, March 2024.
- Li, Y., Tian, H., Ye, D., et al. AutoCodeBench: Large language models are automatic code benchmark generators. *Proceedings of ICLR 2026*, 2025.
- Maheshwari, G., Ivanov, D., and Haddad, K. E. Efficacy of Synthetic Data as a Benchmark, September 2024.
- Mari Oriyad, A., Rohban, M. H., and Soleymani Baghshah, M. The silent judge: Unacknowledged shortcut bias in LLM-as-a-judge. *arXiv preprint arXiv:2509.26072*, 2025.
- Meta AI. The llama 4 herd: The beginning of a new era of natively multimodal AI innovation. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, 2025.
- Mistral AI. Mistral medium 3.5 model card. <https://docs.mistral.ai/models/model-cards/mistral-medium-3-5-26-04>, 2026.
- OpenAI. Introducing GPT-5.4 mini and nano. <https://openai.com/index/introducing-gpt-5-4-mini-and-nano/>, March 2026a.
- OpenAI. GPT-5.5 System Card. <https://openai.com/index/gpt-5-5-system-card/>, April 2026b.

- OpenAI, Hurst, A., Lerer, A., Goucher, A. P., Perelman, A., Ramesh, A., et al. GPT-4o System Card, October 2024.
- Paul, D. G., Zhu, H., and Bayley, I. Benchmarks and Metrics for Evaluations of Code Generation: A Critical Review, June 2024.
- Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27b dense model. <https://qwen.ai/blog?id=qwen3.6-27b>, April 2026.
- Wu, Q., Peng, C., Gao, P., Hu, R., Gan, H., Jiang, B., et al. RepoMasterEval: Evaluating Code Completion via Real-World Repositories, August 2024.
- Yin, J., Ding, H., Luo, J., et al. M2RC-EVAL: Massively multilingual repository-level code completion evaluation. *Proceedings of ACL 2025*, 2025.
- Yin, P., Deng, B., Chen, E., Vasilescu, B., and Neubig, G. Learning to Mine Aligned Code and Natural Language Pairs from Stack Overflow, May 2018.
- Yu, H., Shen, B., Ran, D., Zhang, J., Zhang, Q., Ma, Y., et al. CoderEval: A Benchmark of Pragmatic Code Generation with Generative Pre-trained Models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13, February 2024. doi: 10.1145/3597503.3623322.
- Zhou, S., Alon, U., Agarwal, S., and Neubig, G. CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13921–13937, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.859.
- Zhuo, T. Y., Vu, M. C., Chim, J., Hu, H., Yu, W., Widyasari, R., et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions, April 2025.

A. Benchmark construction details

A.1. Telemetry derivation details

Category derivation proceeded in three stages. First, we drew a stratified, difficulty-enriched sample of 50,000 completions from the full telemetry corpus, balanced across the six target languages and weighted toward completions with low acceptance rates or high post-acceptance edit distances, since these represent scenarios where current models struggle. Sampling weights were used when estimating prevalence, while the balanced sample ensured adequate annotation coverage for lower-volume languages. Each

sampled completion was tagged with telemetry signals including acceptance outcome (accepted as-is, edited, or rejected), character-level edit distance between the generated and final accepted completion, time-to-decision, and completion length. Second, three researchers independently annotated a subset of 3,000 completions (500 per language) with scenario and failure-mode labels (e.g., specialized API usage, low-context completion, pattern extension, incorrect API call, wrong control flow, missing edge case, misunderstood intent, documentation mismatch, syntax error). Inter-annotator agreement on the primary scenario/failure-mode label was substantial (Fleiss’ $\kappa = 0.74$ before adjudication), indicating that the telemetry-derived categories were not artifacts of a single annotator’s judgment. We clustered these annotations based on label co-occurrence, then consolidated the resulting clusters into 8–12 candidate categories per language before cross-language consolidation. Third, the research team consolidated cross-language clusters into the final six categories through iterative discussion with language specialists, merging clusters that shared underlying capability requirements and splitting those that conflated distinct skills. Categories were retained only if they met two criteria: *prevalence* (the scenario or failure family accounted for $\geq 5\%$ of low-acceptance completions across at least four of six languages) and *actionability* (the scenario could be isolated into a self-contained evaluation instance solvable from prefix and suffix alone).

The language distribution in our telemetry corpus is not uniform: Python and JavaScript together account for approximately 55% of completions, while C++ and C# each represent approximately 8–10%. We chose equal representation (300 instances per language) to ensure adequate statistical power for cross-language comparison, rather than mirroring the telemetry distribution. The category distribution in the final benchmark (50 instances per language–category cell) similarly prioritizes balanced evaluation over telemetry frequency.

DevBench offers higher complexity and realism than prior benchmarks, with evaluation instances averaging 57.6 LOC. Importantly, **DevBench** maintains a balanced prefix-to-completion ratio: completions average 11.7 LOC, with 117.9 tokens in the prefix and 108.2 in the completion. In contrast, CrossCodeEval features long prompts (71–116 LOC) but extremely short completions (1–2 LOC). This balance makes **DevBench** more reflective of practical code-completion workflows, where both context and generated code contribute meaningfully to task complexity.

A.2. Benchmark generation and validation details

Each generator produced 684 candidate instances, corresponding to 19 candidates for each of the 36 language-category cells, using the same structured prompts with tem-

perature 0.7 and a 4000-token limit, yielding 2,052 total candidates. Of these, 89 failed automatic syntax or execution validation, 41 were removed by near-duplicate filtering against public benchmarks, and 122 were rejected during blinded human review, leaving 1,800 instances in the final benchmark.

Each instance consists of four components: (1) a **prefix** providing the preceding code context, (2) a **golden completion** as the expected model output, (3) a **suffix** representing subsequent code, and (4) **assertions** to validate correctness. Assertions are stored in a separate field hidden from the model for all six languages, ensuring that models cannot reverse-engineer correct completions from visible test cases. **DevBench** covers both prefix-only completions and fill-in-the-middle (FIM) cases where a suffix is provided. Completions are positioned at natural code boundaries (e.g., after operators, function calls, variable declarations) reflecting realistic developer cursor positions.

Each instance was independently reviewed by two annotators from a team of three senior researchers and engineers with expertise across all six target languages. The annotators evaluated four dimensions: (1) *usefulness* (if the completion satisfies a plausible developer need), (2) *realism* (if it reflects authentic coding patterns, including common inconsistencies and suboptimal but valid approaches), (3) *category alignment* (if it is consistent with the intended task type), and (4) *complexity authenticity* (if it captures the genuine difficulty and edge cases observed in telemetry). Inter-annotator agreement on the initial binary accept/reject decision, before adjudication, was substantial (Cohen’s $\kappa = 0.82$, raw agreement 91%); disagreements were resolved through discussion with the third annotator.

Annotators were specifically instructed to prioritize realism over idealized implementations. For example, API Usage cases were validated not only for correct library calls but also for realistic parameter handling, error conditions, and incomplete context that developers actually encounter in practice. The rejected samples, primarily due to low challenge or category mismatch, were regenerated and re-verified until they met realism standards. Common rejection reasons included: overly simplified or “textbook-perfect” implementations (32% of rejections), insufficient complexity relative to telemetry-observed patterns (28%), unrealistic examples that ignored common edge cases or error conditions (23%), and category misalignment where the completion didn’t match the intended task type (17%). Retention rates were similar across generator families, suggesting that no single generator dominated the final benchmark.

A.3. Generator bias and contamination analysis

Generator-family favoritism. A benchmark generated by a single LLM may contain surface patterns that favor models

from the same provider family. To test this directly, we generated 684 candidate instances per generator using three earlier-generation models from different families (GPT-4o, Claude 3.5 Sonnet, DeepSeek-V3). We fit a logistic regression predicting pass/fail for each model–instance pair, with fixed effects for evaluated model, generator family, language, category, and completion-length bin, and an indicator for whether the evaluated model and generator share a provider family; confidence intervals were estimated by bootstrap resampling over instances. The same-family coefficient was small and not statistically significant. Model rankings computed separately on each generator’s instance subset were also highly correlated, with minimum pairwise Spearman $\rho = 0.91$ across the three generator subsets, and retention rates after validation, near-duplicate filtering, and human review were comparable across generators (GPT-4o: 87.3%, Claude 3.5 Sonnet: 87.9%, DeepSeek-V3: 88.0%). We find no evidence of systematic generator-family favoritism, though we do not claim that all generator effects are absent (Maheshwari et al., 2024; Chen et al., 2024).

Distributional overlap. A broader concern is that LLM-generated code may reflect patterns present in public code corpora. Generator diversity alone cannot eliminate this risk, since modern code models share substantial training-data overlap. **DevBench** therefore uses generation only to instantiate telemetry-derived task structures: the categories, completion modes, cursor positions, difficulty targets, and scenario types are derived from over one billion real developer interactions, while the released code instances are synthetic and privacy-preserving. We audited all candidate instances for near-duplicate overlap against the established benchmarks in Table 3 using token 4-gram Jaccard similarity and AST-level structural similarity. We removed candidates with token 4-gram Jaccard similarity above 0.50 or AST structural similarity above 0.85, thresholds chosen conservatively to flag near-duplicate solutions rather than common programming idioms. Of the 2,052 candidates, 41 (2.0%) exceeded these thresholds and were removed. Manual inspection of all 41 flagged pairs and a random sample of 100 unflagged pairs confirmed that the thresholds primarily captured near-duplicate task structure rather than generic language constructs or common API boilerplate. AST structural similarity is computed over normalized AST node-type sequences after removing identifiers, literals, and formatting.

A.4. Within-prompt diversity analysis

Because each of the 36 language–category prompts generates multiple instances, a concern is whether instances from the same prompt are insufficiently diverse. To test this, we computed average pairwise cosine similarity (using TF-IDF token vectors) among all instances sharing the same prompt. The overall mean pairwise similarity is 0.094,

corresponding to an average cosine distance of 0.906, indicating that instances generated from the same prompt are highly distinct under this token-based measure. Per-language means range from 0.060 (Python, most diverse) to 0.116 (C++, least diverse, reflecting greater boilerplate from includes and namespaces). Per-category means range from 0.077 (API Usage) to 0.117 (Pattern Matching, where shared structural patterns are expected by design). Even the least diverse language–category cell (C# Pattern Matching, 0.159) remains below 16% similarity. These results indicate that prompt specialization does not produce homogeneous instances.

A.5. Benchmark generation prompts

Each language–category cell has a dedicated generation prompt encoding the telemetry-derived task definition, output schema, hidden-assertion format, difficulty requirements, and rejection criteria. All three generator models received the same prompt for a given cell without modification, enabling the cross-generator bias analysis in Section 2.3. Below we reproduce the C++ API Usage prompt as a representative example; the complete set of 36 prompts is released with the benchmark repository.

C++: API Usage Prompts

```
API_USAGE_SYSTEM_PROMPT = """
You are an expert C++ benchmark designer
  creating realistic
code-completion evaluation instances for
  large language models.

Your task is to generate one high-quality C
++ API Usage instance
that reflects telemetry-observed developer
  completion scenarios.
The instance should test whether a model
  can correctly use a
common but error-prone C++ API under
  realistic context
constraints, including
parameter ordering, resource management,
  error handling, type
conversions, API-specific conventions, and
  edge cases.

The instance must be a code-completion task
  , not a standalone
programming problem. The evaluated model
  will see only:
- prefix: code before the cursor
- suffix: optional code after the cursor
  for fill-in-the-middle settings

The evaluated model will NOT see:
- golden_completion
- assertions
- LLM_justification
```

```
Return a single valid JSON object with the
  following fields:
- id: unique identifier
- testsource: "devbench-api-usage"
- language: "cpp"
- prefix: code before the completion
  point
- suffix: code after the completion point
  (may be empty)
- golden_completion: the minimal correct
  code at the cursor
- assertions: hidden executable test code
  used only for
  evaluation; must not duplicate or leak
  the golden completion
- LLM_justification: why this is a
  realistic and challenging
  API Usage task
```

Requirements:

1. Realistic API usage.

Use APIs from the C++ standard library and common development contexts, reflecting telemetry-observed completion scenarios. Representative domains include:

 - Standard algorithms: sort with custom comparators, transform (unary/binary), partition, nth_element, clamp, rotate, search with searcher objects, copy_backward, adjacent_find
 - Smart pointers and memory management: unique_ptr ownership transfer, shared_ptr/make_shared semantics, weak_ptr::lock(), promotion, enable_shared_from_this, allocator patterns
 - Threading and concurrency: async/future, condition_variable with predicates, shared_mutex reader-writer locks, scoped_lock for deadlock-free multi-mutex acquisition, semaphore patterns
 - Container semantics: map insert vs operator[], reserve vs resize, emplace return values, set extract and node handles
 - Move semantics and forwarding: std::move, std::forward for perfect forwarding, std::exchange in move constructors
 - Tuple utilities: std::tie with std::ignore, std::apply for tuple-to-argument expansion, std::tuple_cat
 - Numeric algorithms: accumulate vs reduce, iota, inner_product with custom binary operations
 - String and text processing: substr boundary handling, stoi with index output parameter, string_view lifetime semantics, regex_replace


```

660 - Filesystem: create_directories with
661 error_code overloads,
662 path decomposition and lexical
663 normalization
664 - C++17 vocabulary types: variant with
665 std::visit, optional
666   emplace vs assignment, any_cast value-
667   vs-pointer overloads
668 - Low-level type support: from_chars/
669   to_chars, std::bitset,
670   std::launder with placement new,
671   aligned_storage
672 - Chrono timing: steady_clock vs
673   system_clock, duration_cast
674 - Callable abstractions: std::invoke
675   with member pointers
676 The API call should be embedded in a
677 plausible function, class,
678 or workflow, not presented as isolated
679 trivia.
680
681 2. Difficulty.
682 The completion should require resolving
683 at least two contextual
684 constraints from the prefix/suffix (type
685 compatibility,
686 ownership/lifetime, error-path behavior,
687 parameter ordering,
688 boundary conditions, or consistency with
689 a helper method). The
690 task should not be solvable by copying a
691 nearby line. Avoid
692 textbook-perfect toy examples; include
693 realistic engineering
694 context such as fallback behavior or
695 resource cleanup.
696
697 3. Completion structure.
698 The golden_completion must contain only
699 the code at the cursor.
700 The suffix must not duplicate the
701 golden_completion. The prefix
702 and suffix together must make the
703 completion inferable but not
704 reveal the answer. The combined prefix +
705 golden_completion +
706 suffix + assertions must compile and
707 execute successfully.
708
709 4. Hidden assertions.
710 Assertions must be stored only in the "
711 assertions" field; do
712 not place hidden tests in the prefix or
713 suffix. Assertions must
714 validate functional behavior, not just
715 syntax. Include at least
716 one edge case not explicitly described
717 in comments. Assertions
718 must not hard-code or leak the golden
719 completion.
720
721 5. Rejection criteria. Do not generate
722 instances that are:
723 ambiguous (multiple equally valid
724 completions), dependent on

```

unavailable external services, near-duplicated from public benchmarks, solvable by a single obvious keyword, dominated by boilerplate rather than API reasoning, invalid C++, or overly simplified textbook implementations.

Output a single-line JSON object with all newlines and quotes escaped for JSONL parsing.

```

API_USAGE_USER_PROMPT = """
Generate one C++ API Usage evaluation
instance. Choose a domain
from the representative list in the system
prompt (standard
algorithms, smart pointers, threading,
containers, move
semantics, tuples, numerics, strings,
filesystem, C++17
vocabulary types, low-level type support,
chrono, or callable
abstractions).

```

CRITICAL JSON FORMATTING REQUIREMENTS:

1. Your response MUST be a syntactically valid JSON object
2. PROPERLY ESCAPE all special characters in strings:
 - Use \" for double quotes inside strings
 - Use \n for newlines, \t for tabs, \\ for backslashes
3. The entire JSON object must be on a SINGLE LINE
4. DO NOT use markdown code blocks in your response

Required JSON fields:

- id: unique numeric identifier
- testsource: "devbench-api-usage"
- language: "cpp"
- prefix: code before the completion point (establishes context)
- suffix: code after the completion point; must be DIFFERENT from the golden_completion; may contain execution code but NOT hidden assertions
- golden_completion: the minimal correct code at the cursor that maintains consistency with prefix/suffix
- assertions: hidden executable test code used ONLY for evaluation; the model under test will never see this field; must validate functional behavior and include at least one edge case not spelled out in comments
- LLM_justification: why this is a realistic, challenging task

CRITICAL: HIDDEN ASSERTION REQUIREMENTS:

1. All functional test code must go in the "assertions" field
2. The "assertions" field must NOT be empty
3. Do NOT place hidden tests in the prefix or suffix
4. Assertions must not hard-code or leak the golden completion
5. The combined prefix + golden_completion + suffix + assertions must compile and execute successfully
6. Include at least one edge case assertion

COMPLETION STRUCTURE REQUIREMENTS:

1. The golden_completion must contain ONLY the code at the cursor
2. The suffix must NOT duplicate the golden_completion
3. The prefix and suffix must make the completion inferable but not reveal the answer
4. The completion should require resolving at least two contextual constraints from prefix/suffix

PREFIX LENGTH REQUIREMENTS:

1. The prefix MUST be at least 15-25 lines of code
2. Provide sufficient context and setup code
3. Include helper functions, type definitions, or related code
4. The prefix should demonstrate an incomplete implementation

CODE STRUCTURE REQUIREMENTS:

1. All code must be within proper C++ scope boundaries
2. Do not place executable statements or assertions at global scope; includes, type aliases, constants, class/struct definitions, and helper function definitions are allowed
3. All code blocks must have matching braces
4. Include only headers that are actually used
5. The code must be fully executable C++

INDENTATION REQUIREMENTS:

1. All code sections must maintain consistent indentation
2. The golden_completion must match the prefix indentation level
3. The suffix must maintain the same indentation context

Format your response as a single-line JSON object:

```
{
  "id": "1",
  "testsource": "devbench-api-usage",
  "language": "cpp",
  "prefix": "...",
  "suffix": "...",
  "golden_completion": "...",
  "assertions": "...",
  "LLM_justification": "..."
}
```

```
"...",
"LLM_justification": "..."}
```

VALIDATION CHECKLIST:

1. Is your response a single, valid JSON object?
2. Are all special characters properly escaped?
3. Are assertions in the "assertions" field (NOT in the suffix)?
4. Does prefix + golden_completion + suffix + assertions compile?
5. Does the golden_completion resolve multiple context constraints?
6. Is the task non-trivial and not solvable by copying a line?
7. Are all required JSON fields present, with non-empty prefix, golden_completion, assertions, and LLM_justification? The suffix may be empty for prefix-only instances.
8. Does at least one assertion test an edge case?

B. Evaluation methodology details

B.1. Infrastructure

Our benchmark generation and evaluation workloads were distributed across cloud-based model APIs and local computing resources. Model API calls were orchestrated from a standard laptop (11th Gen Intel i7-1165G7 @ 2.80GHz with 16GB RAM) running Python 3.10. For benchmark generation, using three generator APIs (OpenAI GPT-4o, Anthropic Claude 3.5 Sonnet, and DeepSeek-V3) to create synthetic evaluation instances required approximately 6–12 hours of total wall-clock time across all generators and languages, depending on API latency and excluding human review time. Each individual model evaluation on the complete benchmark required approximately 1.5-3 hours of wall-clock time, also dependent on the API latency. The execution component of our evaluation pipeline, which verifies functional correctness, was executed on the same laptop and required approximately 15 minutes per model (details in Appendix B.2).

B.2. Functional Correctness Evaluation

Our functional correctness evaluation methodology implements robust, secure, and reproducible execution environments across all six programming languages. Each evaluation instance consists of four components: a context prefix, a golden completion (or model-generated completion during evaluation), a context suffix, and assertion statements that verify correctness. The execution pipeline combines these components into complete, executable programs with

language-specific safeguards and dependency management.

Python Execution Environment. Python evaluation instances run in controlled subprocesses with 30-second timeouts to prevent infinite loops. We automatically insert `matplotlib` non-interactive backend configuration to prevent `plt.show()` calls from blocking execution, and handle environment variables securely to provide necessary API access while maintaining isolation. When dependency-related errors occur, our system automatically attempts to install missing packages using `pip` before retrying execution. Each evaluation instance runs in its own isolated environment to prevent cross-contamination between tests.

Java Execution Environment. Java evaluation uses adaptive compilation strategies based on code complexity. Simple test cases without external dependencies use direct `javac` compilation and execution with assertions enabled via the `-ea` flag. Complex cases requiring external libraries (Apache Commons, Jackson, Guava, etc.) automatically utilize Gradle build management with Maven Central dependency resolution. Our system detects package declarations and import statements to determine the appropriate compilation strategy, ensuring both basic and enterprise-level Java code can be properly evaluated.

JavaScript and TypeScript Execution. JavaScript evaluation uses Node.js execution with automatic `npm` package installation for missing dependencies. We configure execution environments with proper `PATH` resolution to ensure consistent Node.js and `npm` access across different system configurations. TypeScript evaluation adds a compilation step using `tsc` before JavaScript execution, with automatic installation of TypeScript compiler and type definitions (`@types` packages) as needed. Both environments support up to five retry attempts for dependency resolution to handle multiple missing packages.

C# Execution Environment. C# evaluation employs `dotnet run` for basic console applications and full MS-Build project compilation for complex scenarios requiring NuGet packages. Our system automatically detects namespace declarations and external dependencies (Entity Framework, Newtonsoft.Json, Azure SDKs, etc.) to generate appropriate `.csproj` files with package references. The execution environment targets .NET 6.0 for broad compatibility while supporting modern C# language features.

C++ Execution Environment. C++ evaluation uses multiple compiler detection (`g++`, `clang++`) with comprehensive library path resolution for external dependencies. Our system automatically detects common libraries (OpenSSL, Boost, OpenCV, Eigen, etc.) from include statements and configures appropriate compiler flags, include paths, and library linking. We support both Homebrew and system-installed libraries across macOS and Linux platforms, with

automatic detection of architecture-specific paths (Apple Silicon vs Intel).

Cross-Language Safeguards. All execution environments implement consistent safety measures: isolated temporary directories with automatic cleanup, configurable timeouts (30-60 seconds based on language compilation requirements), comprehensive error handling with detailed diagnostic reporting, and proper resource management to prevent system interference. Dependencies are installed locally within test directories rather than globally to maintain system isolation.

This multi-language execution infrastructure allows us to comprehensively evaluate functional correctness across diverse programming paradigms while maintaining the security and reproducibility essential for reliable benchmarking. The entire evaluation pipeline generates both human-readable reports and structured JSON output for detailed analysis of model performance across languages and categories.

B.3. Evaluation prompt

For our model evaluation process, we implemented a carefully designed prompt template focused on precise code completion tasks. After initial experimentation revealed that different prompt formats could significantly impact model performance due to formatting issues, we selected a structured instruction-based approach that addresses common failure modes. Our code repository contains the full evaluation prompt.

The selected prompt format provides clear examples demonstrating proper replacement behavior in various scenarios, explicitly instructing models to maintain correct indentation and avoid duplicating existing code structures. By standardizing the input format with clear `#TODO: You Code Here` markers and providing explicit instructions against common mistakes, we created a more level evaluation environment that better isolates models' code understanding capabilities from prompt interpretation abilities.

This evaluation prompt design aligns with real-world code completion scenarios where maintaining contextual formatting is essential for functional correctness, ensuring our benchmark more accurately reflects models' practical utility in development environments. Performance differences observed between models using this standardized prompt more reliably indicate their intrinsic code completion capabilities rather than their ability to navigate ambiguous or unstructured prompting patterns.

B.4. LLM-judge calibration and failure analysis

Judge calibration proceeded in two stages. First, we iteratively tuned the judge prompt on telemetry acceptance

signals from 10,000 completions (accepted as-is, rejected, edited after acceptance), selecting relevance and helpfulness criteria that best separated accepted-as-is, edited, and rejected completions. Low scores correspond to completions that are syntactically plausible but irrelevant to the surrounding context, fail to advance the intended task, omit required edge-case handling, or contradict the prefix/suffix constraints. Second, we validated on a held-out, stratified set of 450 completions (75 per language, sampled across models, categories, and score ranges) with three experienced annotators independently scoring each completion on a 0–10 rubric. The 450-completion validation is intended as a held-out audit of rank-order alignment rather than as a replacement for exhaustive human evaluation; the primary calibration signal comes from the 10,000-completion telemetry tuning set.

For each model, we compute average scores by programming language and evaluation scenario. We then aggregate completions within each language to obtain an overall average and a 95% confidence interval, estimated via 10,000 bootstrap resamples. Finally, we report the overall average score across all languages and completions with its corresponding confidence interval.

Of the 450 validation completions, 50 (11%) exhibited disagreements exceeding 2 points between the averaged human score and the Gemini 2.5 Flash judge score. We manually categorized these into two failure directions.

Judge scores too high (31 of 50 cases). The most common pattern (19 cases) involved completions that were syntactically well-formed and contextually plausible but contained subtle logical errors (such as off-by-one boundary conditions, incorrect operator precedence, or wrong variable references) that human annotators identified from the surrounding context but the judge, which cannot execute code, did not penalize. The remaining 12 cases involved over-generation: the model produced additional code beyond what the task required (e.g., unrequested helper functions or extra validation branches) that the judge rewarded for apparent helpfulness while human annotators penalized for deviating from the expected completion scope.

Judge scores too low (19 of 50 cases). The majority (13 cases) occurred in Code2NL/NL2Code instances, where the judge weighted documentation completeness more heavily than human annotators, penalizing terse but accurate comments that omitted optional tags (e.g., missing `@param` entries in an otherwise correct Javadoc block). The remaining 6 cases involved completions using unconventional but functionally correct idioms (such as ternary chains instead of if/else blocks, or compact list comprehensions instead of explicit loops) that the judge rated as less readable despite human annotators scoring them as acceptable.

These patterns suggest that the judge’s primary vulnerability is an inability to verify functional correctness (scoring too high on plausible-looking but buggy code) and a tendency toward stricter documentation standards than human annotators apply in practice (scoring too low on terse but accurate completions). Both failure modes are consistent with the judge evaluating relevance and helpfulness without execution feedback rather than executable correctness, which is why we report Pass@1 as a complementary metric.

C. Full experimental results

C.1. Full similarity metrics by category

We provide the complete similarity-based results across all evaluated models in Table 7. This expanded view offers a more comprehensive comparison of model performance across different categories and similarity dimensions.

C.2. Pass@1 Performance by Programming Language

Table 8 provides a detailed breakdown of Pass@1 performance across all six programming languages for each evaluated model. Python remains the easiest language for most models (GPT-5.5: 53.6%, Claude Opus 4.7: 54.1%), while the hardest language varies by model: C# is hardest for 4 models, C++ for 2, and TypeScript for 3. DeepSeek V4 Pro is the most balanced across languages, with only a 9.4-percentage-point spread (36.1–45.5%), compared to Claude Opus 4.7’s 44.2-point spread (9.9–54.1%). One model shows a severe C# weakness: Claude Opus 4.7 at 9.9%, despite scoring competitively in all other languages. This reinforces that cross-language evaluation reveals capability gaps invisible in Python-only benchmarks.

C.3. Category correlation analysis

Table 9 reports pairwise Spearman correlations of Pass@1 rates across all 54 model–language combinations (9 evaluated models $\times$ 6 languages).

These correlations are computed over aggregate category Pass@1 scores, so they should be interpreted as an empirical diagnostic of category relationships rather than as a formal latent-factor model of code-generation ability.

Several pairs show low correlation: API Usage vs. Code Purpose Understanding ($\rho = 0.16$) and API Usage vs. Pattern Matching ($\rho = 0.17$), reflecting that library-specific knowledge draws on different capabilities than semantic reasoning or pattern extension over existing code. Conversely, Pattern Matching and Code Purpose Understanding correlate more highly ($\rho = 0.74$), consistent with both requiring semantic reasoning; however, these categories still diverge diagnostically for individual models. This analysis is intended to characterize empirical category relationships, not to prove

Table 7. Similarity metrics across task categories. Most models use reasoning/thinking capabilities; Llama 4 Maverick and Qwen3.6-27B are non-reasoning models.

Model	Average Cosine Similarity						Line 0 Exact Match Rate (%)					
	API Usage	Code2NL NL2Code	Purpose Underst.	Low Context	Pattern Matching	Syntax Compl.	API Usage	Code2NL NL2Code	Purpose Underst.	Low Context	Pattern Matching	Syntax Compl.
GPT-5.5	0.67	0.51	0.42	0.65	0.56	0.67	46.67	48.00	39.00	42.00	44.33	53.33
DeepSeek V4 Pro	0.67	0.56	0.76	0.67	0.78	0.71	46.33	53.00	64.33	44.00	58.00	54.33
Claude Opus 4.7	0.60	0.47	0.48	0.52	0.50	0.61	39.67	40.67	44.33	36.33	38.33	47.00
Llama 4 Maverick	0.63	0.56	0.74	0.67	0.79	0.72	40.00	45.67	51.33	29.67	46.67	42.00
Claude Sonnet 4.6	0.59	0.44	0.29	0.48	0.32	0.53	38.00	43.00	31.00	30.33	26.33	39.67
Mistral Medium 3.5	0.54	0.42	0.63	0.60	0.70	0.61	29.67	35.00	43.33	30.33	43.67	35.67
GPT-5.4 Mini	0.63	0.52	0.23	0.47	0.33	0.40	43.67	46.00	27.00	33.00	36.33	37.33
GPT-5.4 Nano	0.55	0.48	0.15	0.40	0.22	0.40	43.67	48.00	19.67	27.00	24.33	36.67
Qwen3.6-27B	0.44	0.40	0.28	0.36	0.42	0.30	41.00	45.33	36.00	28.67	37.00	32.67

Table 8. Pass@1 rates by programming language using $n = 5$ samples (sorted by overall Pass@1 $\downarrow$). Most models use reasoning/thinking capabilities; Llama 4 Maverick and Qwen3.6-27B are non-reasoning models.

Model	Python	JavaScript	TypeScript	Java	C++	C#
GPT-5.5	53.6%	49.2%	38.9%	47.2%	38.3%	33.7%
DeepSeek V4 Pro	45.2%	45.5%	36.1%	44.5%	45.0%	43.7%
Claude Opus 4.7	54.1%	47.5%	42.5%	47.8%	41.5%	9.9%
Llama 4 Maverick	41.6%	30.9%	30.5%	37.5%	35.9%	38.6%
Claude Sonnet 4.6	44.5%	36.7%	32.0%	36.0%	27.9%	18.6%
Mistral Medium 3.5	44.6%	35.5%	33.5%	38.9%	40.1%	0.0%
GPT-5.4 Mini	31.5%	34.2%	21.6%	28.8%	23.2%	29.9%
GPT-5.4 Nano	30.5%	29.3%	23.8%	27.5%	23.7%	28.1%
Qwen3.6-27B	22.4%	20.3%	14.7%	19.6%	13.5%	17.7%

Table 9. Pairwise Spearman correlations of Pass@1 across categories ($n = 54$ model-language pairs).

	API	Code2NL	Purpose	Low Ctx	Pattern	Syntax
API Usage	1.00					
Code2NL	0.58	1.00				
Purpose	0.16	0.38	1.00			
Low Context	0.36	0.48	0.54	1.00		
Pattern	0.17	0.37	0.74	0.58	1.00	
Syntax	0.36	0.48	0.36	0.60	0.46	1.00

each model-language pair.

that the categories are mutually exclusive; real developer tasks naturally blend capabilities, and correlated categories still provide diagnostic value when their rankings diverge for a specific model.

C.4. Detailed LLM-judge experimental results

Figures 3 and 4 show the breakdown of LLM-judge scores by category and language.

To better understand the relative performance of different models across programming languages, Table 10 presents the LLM-judge scores with 95% confidence intervals for

DevBench: An Interaction-Grounded Benchmark for Code Completion Models

Table 10. Programming language LLM-judge scores of different LLMs with 95% confidence intervals (sorted by overall judge score ↓).

Model	C++	C#	Java	JavaScript	Python	TypeScript
Mistral Medium 3.5	9.23 (9.13-9.33)	9.26 (9.16-9.36)	9.19 (9.09-9.29)	9.20 (9.09-9.30)	9.28 (9.18-9.38)	9.17 (9.06-9.28)
Llama 4 Maverick	9.16 (9.06-9.26)	9.10 (9.00-9.21)	9.13 (9.02-9.24)	9.10 (8.98-9.21)	9.12 (9.01-9.23)	9.16 (9.05-9.26)
DeepSeek V4 Pro	9.70 (9.63-9.76)	9.75 (9.69-9.80)	9.02 (8.87-9.16)	5.42 (5.18-5.67)	6.22 (5.99-6.47)	6.17 (5.93-6.41)
GPT-5.5	6.14 (5.90-6.38)	6.16 (5.92-6.39)	6.08 (5.83-6.32)	6.03 (5.79-6.27)	6.05 (5.80-6.29)	6.21 (5.97-6.45)
GPT-5.4 Nano	5.85 (5.61-6.10)	6.12 (5.88-6.36)	6.33 (6.09-6.56)	6.06 (5.82-6.29)	6.08 (5.85-6.32)	6.00 (5.76-6.24)
Claude Opus 4.7	6.06 (5.82-6.30)	5.85 (5.61-6.10)	6.16 (5.92-6.40)	6.23 (5.99-6.47)	6.00 (5.75-6.24)	6.11 (5.87-6.35)
GPT-5.4 Mini	5.93 (5.69-6.16)	6.10 (5.86-6.34)	6.30 (6.06-6.54)	6.05 (5.81-6.30)	5.90 (5.65-6.14)	6.10 (5.86-6.34)
Claude Sonnet 4.6	7.51 (7.29-7.72)	7.54 (7.33-7.76)	5.71 (5.47-5.95)	4.93 (4.68-5.17)	4.93 (4.68-5.17)	5.17 (4.93-5.41)
Qwen3.6-27B	5.15 (4.91-5.38)	5.06 (4.81-5.30)	5.41 (5.16-5.65)	6.06 (5.82-6.30)	6.03 (5.78-6.28)	6.12 (5.88-6.36)

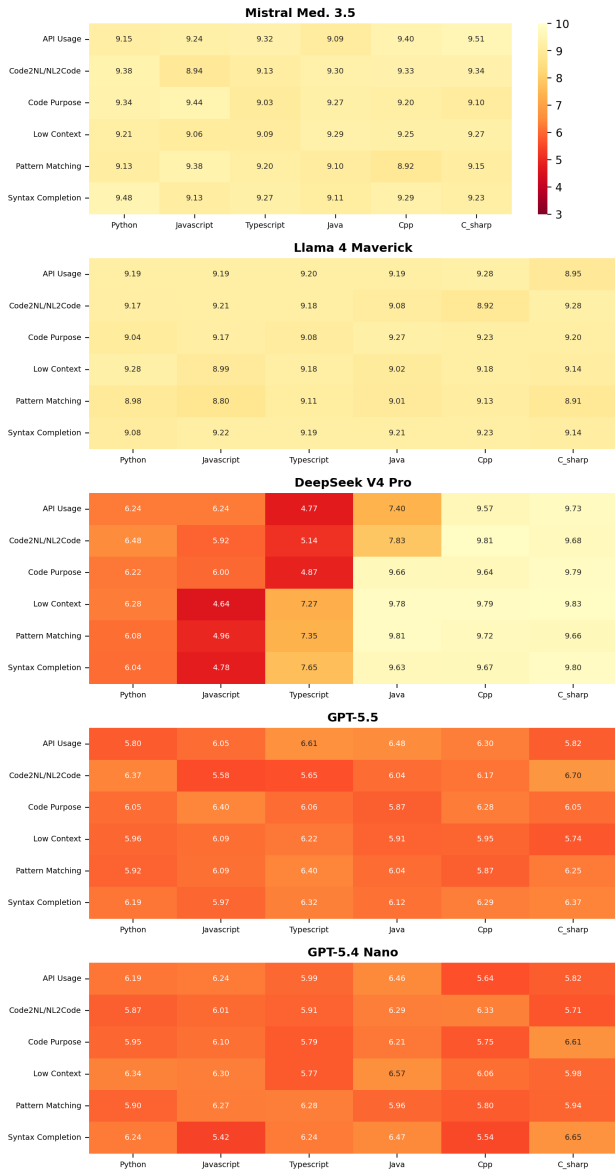


Figure 3. LLM-judge scores by category and language for the top five models (sorted by overall judge score).

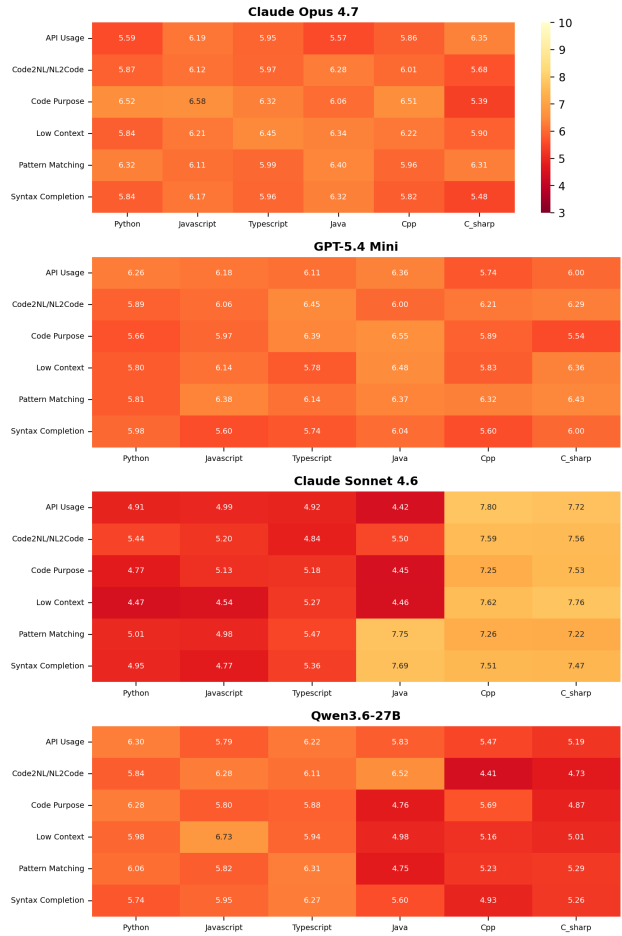


Figure 4. LLM-judge scores by category and language for the remaining four models (continued from Figure 3).

D. Qualitative analysis

D.1. Further category examples

D.1.1. API USAGE

This example tests knowledge of C# 8.0's Index and Range operators introduced for expressive array slicing. The prefix declares an integer array and a comment directing the model to use `^` (index from end) and `..` (range) operators. The golden completion demonstrates six distinct uses: `data[^1]` for the last element, `data[^2]` for second-to-last, `data[2..6]` for a middle slice, `data[^3..]` for the last three elements, `data[..2]` for the first two, and `data[^4..^1]` for a reversed-index range. The suffix validates each slice's content, while hidden assertions test edge cases including `data[^8]` (first element via full-length index), `data[..]` (full copy), and `data[3..3]` (empty range). Models must understand both the `^` from-end semantics and the half-open `..` range semantics simultaneously.

Example 1: C# API Usage #43

```
using System;
namespace Bench
{
    public class Program
    {
        static void Main()
        {
            int[] data = { 10, 20, 30, 40,
                          50, 60, 70, 80 };

            // Use Index (^) and Range (..)
            // operators to extract slices

            int last = data[^1];
            int secondLast = data[^2];
            int[] middle = data[2..6];
            int[] lastThree = data[^3..];
            int[] firstTwo = data[..2];
            int[] reversed = data[^4..^1];
        }
    }
}
```

Golden Completion

```
        if (!(last == 80)) throw new
Exception("last");
        if (!(secondLast == 70)) throw
new Exception("secondLast");
        if (!(middle.Length == 4 &&
middle[0] == 30 && middle[3] == 60))
throw new Exception("middle");
        if (!(lastThree.Length == 3 &&
lastThree[0] == 60)) throw new
Exception("lastThree");
        if (!(firstTwo.Length == 2 &&
firstTwo[1] == 20)) throw new Exception
("firstTwo");
```

```
        if (!(reversed.Length == 3 &&
reversed[0] == 50 && reversed[2] == 70)
) throw new Exception("reversed");
        Console.WriteLine("All slices
correct");
    }
}
```

Suffix

D.1.2. CODE2NL/NL2CODE

This example evaluates the NL-to-Code direction: the model must implement a thread-safe lazy singleton from a detailed Javadoc specification. The prefix provides the complete specification (double-checked locking with a `volatile` field, fast-path null check, synchronized block, and second null check) along with the field declarations and constructor. The golden completion implements `getInstance()` following the spec exactly: outer null check, synchronized (`C2NTask19.class`), inner null check, and instance creation. Hidden assertions verify singleton identity and use reflection to confirm the instance field is declared `volatile`, a subtle but critical requirement that prevents instruction reordering on multicore processors. Models that skip the outer null check (unnecessary synchronization), omit the inner null check (race condition), or forget `volatile` (memory visibility bug) all fail.

Example 2: Java Code2NL/NL2Code #19

```
public class C2NTask19 {

    /**
     * Thread-safe lazy singleton using
     * double-checked locking.
     * The instance field MUST be declared
     * volatile to prevent
     * instruction reordering. The
     * getInstance() method must:
     * 1. Check if instance is null
     * without synchronization (fast path)
     * 2. If null, synchronize on the
     * class object
     * 3. Check again inside the
     * synchronized block (double-check)
     * 4. Create instance only if still
     * null
     * The constructor is private and sets
     * initialized=true.
     */
    private static volatile C2NTask19
instance;
    private final boolean initialized;

    private C2NTask19() {
        this.initialized = true;
    }
}
```

```

1045 public boolean isInitialized() { return
1046     initialized; }
1047
1048 public static C2NTask19 getInstance() {
1049
1050     if (instance == null) {
1051         synchronized (C2NTask19.class)
1052     {
1053         if (instance == null) {
1054             instance = new
1055             C2NTask19();
1056         }
1057     }
1058     return instance;

```

Golden Completion

```

1059
1060
1061 }
1062
1063 public static void main(String[] args)
1064 {
1065     C2NTask19 s1 = C2NTask19.
1066     getInstance();
1067     C2NTask19 s2 = C2NTask19.
1068     getInstance();
1069     assert s1 == s2 : "must be same
1070     instance";
1071     assert s1.isInitialized() : "must
1072     be initialized";
1073     System.out.println("Task 19 PASS");
1074 }

```

Suffix**D.1.3. LOW CONTEXT**

This example tests deep JavaScript knowledge from minimal context. The prefix contains only a comment and a function signature (3 lines total), yet the model must know that JavaScript's Proxy object with a has trap intercepts the in operator, and that Number() coercion is needed to handle string property keys. The golden completion creates a Proxy with has and get traps: has checks numeric membership in a range, get returns the bounds. Hidden assertions verify negative ranges, boundary values, and string-number coercion ('3' in r2 must return true). At only 13 total lines, this task demonstrates that Low Context completion requires recognizing language-specific patterns from minimal cues.

Example 3: JavaScript Low Context #16

```

1094 // Creates a range object where 'in'
1095 // operator checks numeric membership
1096 // and .min/.max properties return bounds
1097 function createRange(min, max) {
1098
1099     return new Proxy({}, {

```

```

    has(_, p) { const n = Number(p);
    return !isNaN(n) && n >= min && n <=
    max; },
    get(_, p) { if (p === 'min') return
    min; if (p === 'max') return max; }
    });

```

Golden Completion

```

}
const r = createRange(1, 10);
const assert = require('assert');
assert.strictEqual(5 in r, true);
assert.strictEqual(15 in r, false);
assert.strictEqual('abc' in r, false);
assert.strictEqual(r.min, 1);
assert.strictEqual(r.max, 10);
console.log('Task 16 passed');

```

Suffix**D.1.4. SYNTAX COMPLETION**

This example tests C++ template syntax with partial specialization. The prefix defines a primary element_type<T> trait that returns T by default, followed by a comment requesting the std::vector specialization. The golden completion writes template <typename T, typename Alloc> struct element_type<std::vector<T, Alloc>> with nested angle brackets, syntactically demanding because the model must correctly nest <T, Alloc> inside <std::vector<...>>. The suffix provides the std::basic_string specialization as a pattern reference and uses static_assert with std::is_same_v for compile-time verification. No runtime assertions are needed; the compiler itself validates correctness.

Example 4: C++ Syntax Completion #41

```

#include <iostream>
#include <vector>
#include <string>
#include <type_traits>
#include <cassert>

// element_type<T> is a type trait that
// extracts the element type from
// container types. For non-container types
// , it returns the type itself.
//
// Supported containers:
//   std::vector<T, Alloc>          -> T
//   std::basic_string<C, T, A>   -> C (
//   the character type)
//   everything else                -> T
//   itself (identity)
//
// This is useful for writing generic code
// that needs to know what a

```



```

1100 // container holds without depending on a
1101 // specific container type.
1102
1103 // Primary template: non-container types
1104 // map to themselves
1105 template <typename T>
1106 struct element_type {
1107     using type = T; // fallback for non-
1108     // containers
1109 };
1110
1111 // Partial specialization for std::vector

```

```

1111 template <typename T, typename Alloc>
1112 struct element_type<std::vector<T, Alloc>>
1113 {
1114     using type = T;
1115 };

```

Golden Completion

```

1119 // Partial specialization for std::
1120 // basic_string
1121 template <typename CharT, typename Traits,
1122 // typename Alloc>
1123 struct element_type<std::basic_string<CharT
1124 // , Traits, Alloc>> {
1125 //     using type = CharT;
1126 // };
1127
1128 template <typename T>
1129 using element_type_t = typename
1130 // element_type<T>::type;
1131
1132 int main() {
1133     static_assert(std::is_same_v<
1134 // element_type_t<int>, int>);
1135     static_assert(std::is_same_v<
1136 // element_type_t<std::vector<double>>,
1137 // double>);
1138     static_assert(std::is_same_v<
1139 // element_type_t<std::string>, char>);
1140
1141     std::cout << "All static_asserts passed
1142 // !" << std::endl;
1143     return 0;
1144 }

```

Suffix

D.1.5. PATTERN MATCHING

This example tests pattern recognition with a subtle trap. The prefix contains two generator functions (chunkArray and chunkString) sharing identical structure: for (let i = 0; i < X.length; i += size) { yield X.slice(i, i + size); }. The golden completion implements slidingWindow, structurally

similar but with two critical differences: i += step instead of i += size, and i <= arr.length - size instead of i < arr.length. A model that blindly copies the chunking pattern produces non-overlapping windows instead of sliding ones. Hidden assertions test window-equals-array, step-larger-than-1, and empty-when-array-smaller-than-window edge cases.

Example 5: JavaScript Pattern Matching #47

```

function* chunkArray(arr, size) {
    // Non-overlapping chunks
    for (let i = 0; i < arr.length; i +=
    size) {
        yield arr.slice(i, i + size);
    }
}

```

```

function* chunkString(str, size) {
    // Same pattern for strings
    for (let i = 0; i < str.length; i +=
    size) {
        yield str.slice(i, i + size);
    }
}

```

```

function* slidingWindow(arr, size, step =
1) {
    // Overlapping chunks: advance by step
    // (not size)
    for (let i = 0; i <= arr.length - size;
    i += step) {
        yield arr.slice(i, i + size);
    }
}

```

Golden Completion

```

const assert = require('assert');
// Non-overlapping chunks
assert.deepStrictEqual([...chunkArray
    ([1,2,3,4,5], 2)], [[1,2],[3,4],[5]]);
assert.deepStrictEqual([...chunkString('
    abcde', 2)], [['ab'],'cd','e']);

// Sliding window: overlapping with step=1
// by default
const windows = [...slidingWindow
    ([1,2,3,4], 3)];
assert.deepStrictEqual(windows,
    [[1,2,3],[2,3,4]]);
// With step=2
const windows2 = [...slidingWindow
    ([1,2,3,4,5], 3, 2)];
assert.deepStrictEqual(windows2,
    [[1,2,3],[3,4,5]]);
console.log('Task 47 suffix passed');

```

Suffix

D.1.6. CODE PURPOSE UNDERSTANDING

This example evaluates whether a model can infer business logic constraints from a state machine. The prefix defines a Subscription class with VALID_TRANSITIONS (pending→active, active→paused/cancelled, etc.) and a _transition helper that validates transitions and logs them to auditLog. The golden completion implements activate(ts) by calling this._transition('active', ts) and recording this.activatedAt = ts, reusing the existing helper rather than setting state directly. Hidden assertions verify that double-activation from the active state throws an error, and that activating a cancelled subscription throws. This tests whether the model reads the state machine constraints and delegates to the validation logic already present in the codebase.

Example 6: JavaScript Code Purpose Understanding #1

```
const assert = require('assert');

class Subscription {
  static VALID_TRANSITIONS = {
    pending: ['active'],
    active: ['paused', 'cancelled'],
    paused: ['active', 'cancelled'],
    cancelled: []
  };

  constructor(userId, plan) {
    this.userId = userId;
    this.plan = plan;
    this.state = 'pending';
    this.auditLog = [];
    this.activatedAt = null;
    this.pausedAt = null;
    this.totalPausedMs = 0;
  }

  _transition(newState, ts) {
    const allowed = Subscription.
    VALID_TRANSITIONS[this.state];
    if (!allowed || !allowed.includes(
    newState)) {
      throw new Error(`Cannot
    transition from ${this.state} to ${
    newState}`);
    }
    const old = this.state;
    this.state = newState;
    this.auditLog.push({ from: old, to:
    newState, ts });
  }

  activate(ts) {
    this._transition('active', ts);
    this.activatedAt = ts;
  }
}
```

Golden Completion

```
pause(ts) {
  this._transition('paused', ts);
  this.pausedAt = ts;
}

const s1 = new Subscription('u1', 'pro');
s1.activate(1000);
assert.strictEqual(s1.state, 'active');
assert.strictEqual(s1.activatedAt, 1000);
assert.strictEqual(s1.auditLog.length, 1);
assert.deepStrictEqual(s1.auditLog[0], {
  from: 'pending', to: 'active', ts: 1000
});
```

Suffix

D.2. Illustrative model comparison

D.2.1. C++ SYNTAX COMPLETION: GPT-5.5 vs. DEEPSEEK V4 PRO

The prefix defines tuple_sig_impl using C++17 index sequences and fold expressions. The completion must implement the function body AND define a separate tuple_sig wrapper that the suffix's main() calls. GPT-5.5 correctly generates both: a fold expression body using comma-separated type_tag calls, then closes the function and defines the forwarding tuple_sig template. DeepSeek V4 Pro produces a single elegant fold expression return "(" + (... + (...)) + ")" that correctly implements the immediate function logic but never closes the function body or defines tuple_sig. This causes an “undeclared identifier” error when main() calls tuple_sig(t1). The failure pattern is consistent across all 8 non-GPT-5.5 models: each completes only the immediate function body without recognizing that the suffix requires additional definitions. This demonstrates that Syntax Completion requires structural awareness, understanding what identifiers and definitions the surrounding code depends on, not just syntactic knowledge of fold expressions. The example explains the 19-percentage-point gap between GPT-5.5 and DeepSeek V4 Pro on Syntax Completion.

Example 7: C++ Syntax Completion #6

```
#include <type_traits>
#include <tuple>
#include <utility>
#include <string>
#include <cassert>

// Compile-time type name via constexpr if
// + template specialization.
// Returns a string tag for the type.
template<typename T>
std::string type_tag() {
```

```

1210     if constexpr (std::is_same_v<T, int>)
1211     return "i";
1212     else if constexpr (std::is_same_v<T,
1213     double>) return "d";
1214     else if constexpr (std::is_same_v<T,
1215     std::string>) return "s";
1216     else return "?";
1217 }

```

```

1217 // Build a "signature" string from a tuple'
1218 // s types, e.g. "(i,d,s)"
1219 // Uses index_sequence to expand the tuple
1220 // types at compile time.
1221 template<typename Tuple, std::size_t... Is>
1222 std::string tuple_sig_impl(std::
1223     index_sequence<Is...>) {

```

```

1224     std::string result = "(";
1225     int idx = 0;
1226     ((result += (idx++ > 0 ? ", " : "") +
1227     type_tag<std::tuple_element_t<Is,
1228     Tuple>>()), ...);
1229     return result + ")";
1230 }
1231 template<typename... Ts>
1232 std::string tuple_sig(const std::tuple<Ts
1233     ...>&) {
1234     return tuple_sig_impl<std::tuple<Ts
1235     ...>>(
1236     std::index_sequence_for<Ts...>{});

```

Golden Completion

```

1237     std::string sig = "(";
1238     ((sig += (Is == 0 ? "" : ", "), sig +=
1239     type_tag<std::tuple_element_t<Is, Tuple
1240     >>()), ...);
1241     sig += ")";
1242     return sig;
1243 }
1244 template<typename... Ts>
1245 std::string tuple_sig(const std::tuple<Ts
1246     ...>&) {
1247     return tuple_sig_impl<std::tuple<Ts
1248     ...>>(std::index_sequence_for<Ts...>{}))
1249     ;
1250 }

```

Model Completion (GPT-5.5)

```

1253     return "(" + ( ... + (type_tag<std::
1254     tuple_element_t<Is, Tuple>>()) + (Is + 1
1255     == sizeof...(Is) ? "" : ",") ) + ")";

```

Model Completion (DeepSeek V4 Pro)

```

1258 }
1259 int main() {
1260     auto t1 = std::make_tuple(1, 2.0, std::
1261     string("hi"));
1262     assert(tuple_sig(t1) == "(i,d,s)");
1263 }
1264

```

```

    auto t2 = std::make_tuple(42);
    assert(tuple_sig(t2) == "(i)");

    auto t3 = std::make_tuple(1.0, 2.0);
    assert(tuple_sig(t3) == "(d,d)");
    return 0;
}

```

Suffix

D.2.2. TYPESCRIPT CODE PURPOSE: CLAUDE OPUS 4.7 VS. DEEPSEEK V4 PRO

The prefix defines a `BankingService` class with a `TransferAudit` interface specifying `status: 'completed' | 'failed'` and a separate `reason?: string` field. The golden completion implements `transfer()` to return `{ status: 'failed', reason: 'daily_limit_exceeded' }` when validation fails. DeepSeek V4 Pro (and all other models except Opus) instead writes `status: 'daily_limit_exceeded'`, embedding the error type directly into the status field rather than using the interface's two-field pattern. DeepSeek also omits pushing failed transfers to the audit log. Only Claude Opus 4.7 correctly interprets the TypeScript interface structure, producing `status: 'failed'` with the reason in the dedicated field. This demonstrates that Code Purpose Understanding requires reading type definitions as behavioral specifications, not just syntactic constraints.

Example 8: TypeScript Code Purpose Understanding #31

```

interface Account {
    id: string;
    balance: number;
    dailyTransferred: number;
    lastTransferDay: number;
}

```

```

interface TransferAudit {
    from: string;
    to: string;
    gross: number;
    fee: number;
    net: number;
    day: number;
    status: string;
    reason?: string;
}

```

```

class BankingService {
    accounts: Map<string, Account> = new
    Map();
    dailyLimit: number;
    feeRate: number; // fraction, e.
    g. 0.02 = 2%
    minFee: number; // minimum fee
    per transfer
}

```

```

1265   auditLog: TransferAudit[] = [];
1266
1267   constructor(dailyLimit: number, feeRate
: number, minFee: number) {
1268       this.dailyLimit = dailyLimit;
1269       this.feeRate = feeRate;
1270       this.minFee = minFee;
1271   }
1272
1273   addAccount(id: string, balance: number)
: void {
1274       this.accounts.set(id, { id, balance
, dailyTransferred: 0, lastTransferDay:
-1 });
1275   }
1276
1277   // Transfer `amount` from `fromId` to `
toId` on `day`.
1278   // Fee = max(amount * feeRate, minFee),
deducted from SENDER in addition to
amount.
1279   // Total debit = amount + fee.
1280   // Check order:
1281   // 1. Both accounts exist, else '
account_not_found'.
1282   // 2. Reset dailyTransferred if day
differs from lastTransferDay.
1283   // 3. dailyTransferred + amount >
dailyLimit? 'daily_limit_exceeded'.
1284   // 4. Sender balance < amount + fee?
'insufficient_funds'.
1285   // 5. Success: debit sender (amount+
fee), credit receiver (amount only),
// update dailyTransferred, log
audit with gross/fee/net.
1286
1287
1288
1289
1290
1291
1292
1293
1294

```

```

1295   transfer(fromId: string, toId: string,
amount: number, day: number):
TransferAudit {
1296       const from = this.accounts.get(
fromId);
1297       const to = this.accounts.get(toId);
1298       if (!from || !to) {
1299           const audit: TransferAudit = {
1300               from: fromId, to: toId, gross: amount,
1301               fee: 0, net: 0, day, status: 'failed',
1302               reason: 'account_not_found' };
1303           this.auditLog.push(audit);
1304           return audit;
1305       }
1306       if (from.lastTransferDay !== day) {
1307           from.dailyTransferred = 0;
1308           from.lastTransferDay = day;
1309       }
1310       if (from.dailyTransferred + amount
> this.dailyLimit) {
1311           const audit: TransferAudit = {
1312               from: fromId, to: toId, gross: amount,
1313               fee: 0, net: 0, day, status: 'failed',
1314               reason: 'daily_limit_exceeded' };
1315           this.auditLog.push(audit);
1316           return audit;
1317       }
1318       const fee = Math.max(Math.round(
amount * this.feeRate * 100) / 100,
1319

```

```

this.minFee);
    const totalDebit = amount + fee;
    if (from.balance < totalDebit) {
        const audit: TransferAudit = {
            from: fromId, to: toId, gross: amount,
            fee, net: 0, day, status: 'failed',
            reason: 'insufficient_funds' };
        this.auditLog.push(audit);
        return audit;
    }
    from.balance -= totalDebit;
    to.balance += amount;
    from.dailyTransferred += amount;
    const audit: TransferAudit = { from
: fromId, to: toId, gross: amount, fee,
    net: amount, day, status: 'success' };
    this.auditLog.push(audit);
    return audit;
}

```

Golden Completion

```

transfer(fromId: string, toId: string,
amount: number, day: number):
TransferAudit {
    const from = this.accounts.get(
fromId);
    const to = this.accounts.get(toId);
    if (!from || !to) {
        const audit: TransferAudit = {
            from: fromId, to: toId,
            gross: amount, fee: 0, net: 0, day,
            status: 'failed', reason: '
account_not_found'
        };
        this.auditLog.push(audit);
        return audit;
    }

    if (from.lastTransferDay !== day) {
        from.dailyTransferred = 0;
        from.lastTransferDay = day;
    }

    if (from.dailyTransferred + amount
> this.dailyLimit) {
        const audit: TransferAudit = {
            from: fromId, to: toId,
            gross: amount, fee: 0, net: 0, day,
            status: 'failed', reason: '
daily_limit_exceeded'
        };
        this.auditLog.push(audit);
        return audit;
    }

    const fee = Math.max(amount * this.
feeRate, this.minFee);
    if (from.balance < amount + fee) {
        const audit: TransferAudit = {
            from: fromId, to: toId,
            gross: amount, fee, net: amount, day,
            status: 'failed', reason: '
insufficient_funds'
        };
    }
}

```



```

1320         this.auditLog.push(audit);
1321         return audit;
1322     }
1323
1324     from.balance -= amount + fee;
1325     to.balance += amount;
1326     from.dailyTransferred += amount;
1327
1328     const audit: TransferAudit = {
1329         from: fromId, to: toId, gross:
1330         amount, fee, net: amount, day,
1331         status: 'success'
1332     };
1333     this.auditLog.push(audit);
1334     return audit;
1335 }

```

Model Completion (Claude Opus 4.7)

```

1320         reason: 'Insufficient funds
1321     '
1322     };
1323
1324     fromAccount.balance -= totalDebit;
1325     toAccount.balance += amount;
1326     fromAccount.dailyTransferred +=
1327     amount;
1328     const audit: TransferAudit = {
1329         from: fromId, to: toId, gross:
1330         amount, fee, net: amount, day,
1331         status: 'success'
1332     };
1333     this.auditLog.push(audit);
1334     return audit;
1335 }

```

Model Completion (DeepSeek V4 Pro)

```

1336 transfer(fromId: string, toId: string,
1337 amount: number, day: number):
1338 TransferAudit {
1339     const fromAccount = this.accounts.
1340     get(fromId);
1341     const toAccount = this.accounts.get
1342     (toId);
1343     if (!fromAccount || !toAccount) {
1344         return {
1345             from: fromId, to: toId,
1346             gross: amount, fee: 0, net: 0, day,
1347             status: 'account_not_found'
1348         },
1349         {
1350             reason: 'One or both
1351             accounts not found'
1352         };
1353     }
1354     if (fromAccount.lastTransferDay !==
1355     day) {
1356         fromAccount.dailyTransferred =
1357         0;
1358         fromAccount.lastTransferDay =
1359         day;
1360     }
1361     if (fromAccount.dailyTransferred +
1362     amount > this.dailyLimit) {
1363         return {
1364             from: fromId, to: toId,
1365             gross: amount, fee: 0, net: 0, day,
1366             status: '
1367             daily_limit_exceeded',
1368             reason: 'Daily limit
1369             exceeded'
1370         };
1371     }
1372     const fee = Math.max(amount * this.
1373     feeRate, this.minFee);
1374     const totalDebit = amount + fee;
1375     if (fromAccount.balance <
1376     totalDebit) {
1377         return {
1378             from: fromId, to: toId,
1379             gross: amount, fee, net: 0, day,
1380             status: 'insufficient_funds
1381             ',

```

```

1382 }
1383
1384 const assert = require('assert');
1385
1386 const bank = new BankingService(1000, 0.02,
1387 5);
1388 bank.addAccount('A', 500);
1389 bank.addAccount('B', 200);
1390
1391 // Normal transfer: amount=100, fee=max(2,
1392 5)=5, total debit=105
1393 const r1 = bank.transfer('A', 'B', 100, 1);
1394 assert.strictEqual(r1.status, 'success');
1395 assert.strictEqual(r1.fee, 5);
1396 assert.strictEqual(bank.accounts.get('A')!.
1397 balance, 395);
1398 assert.strictEqual(bank.accounts.get('B')!.
1399 balance, 300);
1400
1401 // Large transfer: amount=300, fee=max(6,
1402 5)=6, total=306
1403 const r2 = bank.transfer('A', 'B', 300, 1);
1404 assert.strictEqual(r2.fee, 6);
1405 assert.strictEqual(bank.accounts.get('A')!.
1406 balance, 89);
1407
1408 console.log('Task 31 suffix passed');

```

Suffix

D.3. Qualitative examples

D.3.1. FAILED EXECUTION, DISSIMILAR COMPLETION

The prefix provides a `Result<T>` union type and a `Step<A, B>` type alias. Claude Opus 4.7 generates function overloads, a TypeScript best practice for type-safe pipelines, with signatures for 1, 2, and 3 steps, plus a general implementation signature. However, every overload requires at least one step argument. The hidden assertion pipeline (1) invokes the function with zero steps, which matches none of the overloads and produces TS2554 (“Expected 2–4 arguments, but got 1”). DeepSeek V4 Pro

1375 uses a simpler rest parameter ...steps: Step<any,
1376 any>[] that accepts any number of steps including zero.
1377 This demonstrates a paradox where more sophisticated Type-
1378 Script knowledge (generating type-safe overloads) produces
1379 code that over-constrains the API relative to simpler, more
1380 permissive implementations.

Example 9: TypeScript Low Context #29

```
1384 type Result<T> = { ok: true; value: T } | {  
1385   ok: false; error: string };  
1386 type Step<A, B> = (a: A) => Result<B>;
```

```
1387 function pipeline<T>(input: T, ...steps:  
1388   Step<any, any>[]): Result<any> {  
1389   let current: Result<any> = { ok: true,  
1390     value: input };  
1391   for (const step of steps) {  
1392     if (!current.ok) return current;  
1393     current = step(current.value);  
1394   }  
1395   return current;  
1396 }
```

Golden Completion

```
1398 function pipeline<A, B>(input: A, s1: Step<  
1399   A, B>): Result<B>;  
1400 function pipeline<A, B, C>(input: A, s1:  
1401   Step<A, B>, s2: Step<B, C>): Result<C>;  
1402 function pipeline<A, B, C, D>(input: A, s1:  
1403   Step<A, B>, s2: Step<B, C>, s3: Step<C  
1404   , D>): Result<D>;  
1405 function pipeline(input: any, ...steps:  
1406   Step<any, any>[]): Result<any> {  
1407   let current: Result<any> = { ok: true,  
1408     value: input };  
1409   for (const step of steps) {  
1410     if (!current.ok) return current;  
1411     current = step(current.value);  
1412   }  
1413   return current;  
1414 }
```

Model Completion (Claude Opus 4.7)

```
1415 function pipeline<T>(init: T, ...steps:  
1416   Step<any, any>[]): Result<any> {  
1417   let result: Result<any> = { ok: true,  
1418     value: init };  
1419   for (const step of steps) {  
1420     if (!result.ok) break;  
1421     result = step(result.value);  
1422   }  
1423   return result;  
1424 }
```

Model Completion (DeepSeek V4 Pro)

```
1426 import { strict as assert } from "assert";  
1427 const double: Step<number, number> = n =>  
1428   ({ ok: true, value: n * 2 });  
1429
```

```
const failIfBig: Step<number, number> = n  
=> n > 100 ? { ok: false, error: "too  
big" } : { ok: true, value: n };  
const stringify: Step<number, string> = n  
=> ({ ok: true, value: String(n) });  
assert.deepEqual(pipeline(5, double,  
  stringify), { ok: true, value: "10" });  
assert.deepEqual(pipeline(60, double,  
  failIfBig), { ok: false, error: "too  
big" });
```

Suffix

D.3.2. FAILED EXECUTION, SIMILAR COMPLETION

The prefix establishes helper functions shallowEqual (with epsilon for floats) and arrayEqual (order-independent comparison via sorting). Both GPT-5.5 and DeepSeek V4 Pro implement deepEqual with identical structure: null checks, type checks, recursive object comparison with sorted keys. The critical difference is a single guard clause. GPT-5.5 writes if (Array.isArray(a) || Array.isArray(b)) followed by a nested check if (!Array.isArray(a) || !Array.isArray(b)) return false, which handles the mixed case where one argument is an array and the other is a plain object. DeepSeek writes if (Array.isArray(a) && Array.isArray(b)) which only enters the array branch when both are arrays, letting mixed array/object pairs fall through to object comparison. The hidden assertion deepEqual([], {}) === false exploits this: an empty array and empty object both have zero enumerable keys, so the object-comparison path returns true. This one missing guard (|| vs &&) is the entire difference between 5/5 and 1/5. The example demonstrates how structurally similar completions can contain subtle logical errors that only surface under specific edge cases.

Example 10: JavaScript Pattern Matching #44

```
function shallowEqual(a, b) {  
  if (typeof a !== typeof b) return false  
  ;  
  if (typeof a === 'number') return Math.  
    abs(a - b) < 1e-9;  
  return a === b;  
}
```

```
function arrayEqual(a, b) {  
  // Pattern: arrays are compared as  
  SORTED SETS (order-independent)  
  if (a.length !== b.length) return false  
  ;  
  const sa = [...a].sort((x, y) => JSON.  
    stringify(x) < JSON.stringify(y) ? -1 :  
    1);  
  const sb = [...b].sort((x, y) => JSON.  
    stringify(x) < JSON.stringify(y) ? -1 :  
    1);
```

```

1430     return sa.every((v, i) => deepEqual(v,
1431     sb[i]));
1432 }
1433
1434 function deepEqual(a, b) {
1435     if (a === b) return true;
1436     if (a == null || b == null) return a
1437     === b;
1438     if (typeof a === 'number' && typeof b
1439     === 'number') return Math.abs(a - b) <
1440     1e-9;
1441     if (typeof a !== typeof b) return false
1442     ;
1443     if (Array.isArray(a) !== Array.isArray(
1444     b)) return false;
1445     if (Array.isArray(a)) return arrayEqual
1446     (a, b);
1447     if (typeof a !== 'object') return false
1448     ;
1449     const ka = Object.keys(a).sort(), kb =
1450     Object.keys(b).sort();
1451     if (ka.length !== kb.length) return
1452     false;
1453     return ka.every((k, i) => k === kb[i]
1454     && deepEqual(a[k], b[k]));
1455 }

```

Golden Completion

```

1454 function deepEqual(a, b) {
1455     if (a === null || b === null) return a
1456     === b;
1457     if (typeof a !== typeof b) return false
1458     ;
1459     if (typeof a !== 'object') return
1460     shallowEqual(a, b);
1461     if (Array.isArray(a) || Array.isArray(b)
1462     ) {
1463         if (!Array.isArray(a) || !Array.
1464         isArray(b)) return false;
1465         return arrayEqual(a, b);
1466     }
1467     const ka = Object.keys(a).sort();
1468     const kb = Object.keys(b).sort();
1469     if (ka.length !== kb.length) return
1470     false;
1471     return ka.every((key, i) => key === kb[
1472     i] && deepEqual(a[key], b[key]));
1473 }

```

Model Completion (GPT-5.5)

```

1475 function deepEqual(a, b) {
1476     if (a === b) return true;
1477     if (a == null || b == null) return a
1478     === b; // handles null/undefined
1479     correctly
1480     if (Array.isArray(a) && Array.isArray(b)
1481     ) {
1482         return arrayEqual(a, b);
1483     }
1484     if (typeof a === 'object' && typeof b
1485     === 'object') {

```

```

const keysA = Object.keys(a);
const keysB = Object.keys(b);
if (keysA.length !== keysB.length)
return false;
return keysA.every(key => deepEqual
(a[key], b[key]));
}
return shallowEqual(a, b);
}

```

Model Completion (DeepSeek V4 Pro)

```

const assert = require('assert');
// Numbers with epsilon
assert.strictEqual(deepEqual(0.1 + 0.2,
0.3), true);
// Arrays as sets (order independent)
assert.strictEqual(deepEqual([1, 2, 3], [3,
1, 2]), true);
assert.strictEqual(deepEqual([1, 2], [2,
3]), false);
// Nested objects
assert.strictEqual(deepEqual({a: 1}, {a:
1}), true);
assert.strictEqual(deepEqual({a: 1}, {a:
2}), false);
// Null handling
assert.strictEqual(deepEqual(null, null),
true);
assert.strictEqual(deepEqual(null, 0),
false);
console.log('Task 44 suffix passed');

```

Suffix

D.3.3. PASSED EXECUTION, DISSIMILAR COMPLETION

The prefix specifies a slugify function that must lowercase input, replace non-alphanumeric characters with hyphens, collapse multiple hyphens, strip leading/trailing hyphens, AND insert hyphens at camelCase boundaries and letter-digit boundaries. The golden completion implements this as a single-pass `StringBuilder` loop: it iterates character-by-character, checking the previous character against the current to detect camelCase transitions (`isLowerCase(p) && isUpperCase(ch)`) and letter-digit boundaries, building the result in one traversal. DeepSeek V4 Pro takes a fundamentally different approach: a pipeline of six `String.replaceAll` calls, each handling one concern: camelCase splitting via `(?<=[a-z])(?=[A-Z])` lookbehind/lookahead, lowercase conversion, digit-boundary splitting, non-alphanumeric replacement, hyphen collapsing, and leading/trailing stripping. The two implementations share almost no tokens (similarity 0.124) yet produce identical output on all test cases. This illustrates why **DevBench** reports both Pass@1 and similarity metrics: functional correctness alone would treat these as equivalent, while similarity reveals they represent genuinely different problem-solving strategies: impera-

tive single-pass parsing versus declarative multi-pass regex transformation.

Example 11: Java Pattern Matching #20

```
public class PMTask20 {
    // Custom slugify: lowercase, non-alnum
    // -> hyphen, collapse hyphens,
    // strip leading/trailing hyphens.
    // ADDITIONALLY: insert hyphen at
    // camelCase boundaries (lower->Upper)
    // AND at letter<->digit boundaries.
    public static String slugify(String
input) {
```

```
    StringBuilder sb = new
StringBuilder();
    for (int i = 0; i < input.length();
i++) {
        char ch = input.charAt(i);
        if (!Character.isLetterOrDigit(
ch)) { if (sb.length() > 0 && sb.charAt
(sb.length() - 1) != '-') sb.append('-'
); continue; }
        if (sb.length() > 0 && sb.
charAt(sb.length() - 1) != '-' && i >
0) {
            char p = input.charAt(i -
1);
            if ((Character.isLowerCase(
p) && Character.isUpperCase(ch)) || (
Character.isLetter(p) && Character.
isDigit(ch)) || (Character.isDigit(p)
&& Character.isLetter(ch))) sb.append('
-');
        }
        sb.append(Character.toLowerCase
(ch));
    }
    String s = sb.toString(); while (s.
startsWith("-")) s = s.substring(1);
    while (s.endsWith("-")) s = s.substring
(0, s.length()-1);
    return s;
```

Golden Completion

```
String s = input;
// Insert hyphen at camelCase
boundaries (lowercase followed by
uppercase)
s = s.replaceAll("(?<=[a-z])(?=[A-Z
])", "-");
// Convert to lowercase
s = s.toLowerCase();
// Insert hyphen at letter-digit
boundaries
s = s.replaceAll("(?<=[a-z])(?=[\d]
|(?<=[\d])(?=[a-z])", "-");
// Replace non-alphanumeric
characters with hyphens
s = s.replaceAll("[^a-z0-9]+", "-")
;
// Collapse multiple hyphens
```

```
s = s.replaceAll("-", "-");
// Strip leading/trailing hyphens
s = s.replaceAll("^-|-$", "");
return s;
```

Model Completion (DeepSeek V4 Pro)

```
}

public static void main(String[] args)
{
    // Custom slugify with camelCase
    and digit boundary splitting
    assert slugify("helloWorld").equals(
"hello-world") : "camelCase";
    assert slugify("item3pack").equals(
"item-3-pack") : "digit boundaries";
    assert slugify("hello world").
equals("hello-world") : "spaces to
hyphens";
    assert !slugify("item3pack").equals(
"item3pack") : "digits must be split";
    assert !slugify("helloWorld").
equals("helloworld") : "camelCase must
split";
    assert slugify(" leading ").
equals("leading") : "strip leading/
trailing";
    System.out.println("OK: " + slugify
("myVar2Go"));
}
```

Suffix

E. Limitations, assumptions, and broader impacts

E.1. Limitations and future directions

While **DevBench** represents a significant advancement in code completion evaluation, we identify two principal directions for future work.

E.1.1. ENHANCING EVALUATION FRAMEWORKS

The complementary evaluation metrics we employ (Pass@1, similarity-based metrics, and LLM-judge assessments) provide multidimensional insights into model performance. The occasional divergence between these metrics, such as cases where higher syntactic similarity does not correlate with functional correctness, highlights an opportunity to develop composite metrics that better capture the full spectrum of code quality dimensions relevant to developers. Future work could also explore ensemble judging approaches combining multiple judge models, or contrastive evaluation techniques that specifically control for stylistic biases.

E.1.2. BROADENING COVERAGE SCOPE

DevBench currently covers six major programming languages and six task categories focused on code completion. Future extensions could apply our telemetry-driven methodology to additional development activities such as code refactoring, debugging, and multi-file architecture design, as well as incorporate emerging languages such as Rust, Go, and Swift.

E.2. Assumptions

The evaluative claims supported by **DevBench** rest on the following assumptions:

1. **Fill-in-the-middle evaluation mode.** Models are evaluated using a structured FIM prompt (Appendix B.3) that presents prefix, a #TODO marker, and suffix. Results may differ under chat-style or agentic prompting, and scores should not be interpreted as measuring general coding ability outside this completion paradigm.
2. **Sampling representativeness.** We report Pass@1 from $n=5$ samples at temperature 0.2 (or default reasoning settings for models that do not expose temperature). Different sampling parameters could shift absolute scores, though we expect relative model rankings to be robust based on prior work showing rank stability across temperature settings (Chen et al., 2021).
3. **Assertion coverage.** Functional correctness is determined by a finite set of hidden assertions per task. These assertions test key behaviors but do not guarantee full specification coverage; a completion that passes all assertions may still contain latent bugs, and one that fails may be correct on dimensions the assertions do not cover. This is precisely why **DevBench** employs a multi-metric evaluation framework: similarity-based metrics capture syntactic closeness to the golden completion regardless of assertion outcomes, and LLM-judge assessments evaluate contextual relevance and helpfulness without relying on execution. Together, the three complementary metrics mitigate the limitations of any single measure.
4. **LLM-judge scope.** The Gemini 2.5 Flash judge evaluates relevance and helpfulness without executing code. Its scores complement but do not substitute for functional correctness; systematic failure modes are documented in Appendix B.4.

E.3. Broader impacts

Our **DevBench** benchmark has several potential positive societal impacts. By enabling more accurate evaluation of code completion models, our work can lead to improved

developer productivity tools that reduce repetitive coding tasks, decrease the time required to implement software solutions, and potentially lower barriers to entry in programming by assisting novice developers. More accurate code completion could also improve software quality by suggesting well-tested patterns and reducing common programming errors, potentially leading to more reliable and secure software systems.

However, we also acknowledge several potential negative impacts. First, there are fairness considerations related to programming language representation; our benchmark’s coverage of six languages, while broader than many existing benchmarks, still represents a limited subset of the programming ecosystem. This may lead to uneven improvements across programming languages, potentially disadvantaging developers who work primarily with languages not included in our benchmark. Second, there are potential job market implications if increasingly capable code completion systems begin to automate significant portions of software development tasks, potentially affecting employment opportunities for certain types of programming roles.

Additionally, we recognize that improvements in code generation capabilities could have security implications. While our benchmark focuses on code completion rather than full program generation, advances in code synthesis could potentially be misused to generate malicious code more efficiently or to exploit vulnerabilities in existing systems. To mitigate these concerns, we have designed our benchmark to emphasize proper API usage, security patterns, and code quality metrics rather than merely measuring functional correctness.

To address these concerns, we have made our benchmark and methodology publicly available to enable community scrutiny, external validation, and continuous improvement. We encourage future research to extend language coverage, develop more diverse evaluation metrics, and carefully monitor potential misuses of increasingly capable code generation systems.